

DOUBLE DESCENT AND STATISTICAL THEORY

Bui Gia Khanh & Luong Van Tam

Department of Physics

Hanoi University of Science, Vietnam National University

Hanoi, Vietnam

{fujimiyaamane}@outlook.com, tam0309cnvl@gmail.com

Dang Tri Trung

Monash University, Department of Computer Science

Melbourne, Australia

trungdc0708@gmail.com

ABSTRACT

The analysis of learning action, machine learning and related practices in the field theoretically has been made by utilizing Computational Learning Theory (CoLT) Valiant (1984) and Statistical Learning Theory (SLT) Vapnik (1999). These two theories, while overlapped, provides a general framework in analysing and justifying learning actions and learning model constructions, aiding in the formation of modern practices. One of the famous insight using such framework is the *bias-variance tradeoff* Geman et al. (1992), which states that model complexity and generality inversely affect each other, thus guarantee the need for a safe bracket between them. However, recent literatures, Belkin et al. (2019) has indicated the fallout of such dilemma by the new phenomenon called *double descent*, where there exists an interpolation threshold that renders the current statistical justification for bias-variance inconsequential to a given region. Various ‘anomaly’ has also been detected similarly, with varying degree of sophistication and potential. In this paper, we would like to review through progress made in this particular field of research, and the phenomena potential explanations up until now.

1 INTRODUCTION

The theory of machine learning and its modern practice has been developed and researched, of a substantial portion by empirical and heuristic approach, either by advancing new practices, architectures or by try-and-test modification. From its early onset of a regression estimator $\theta(x, y)$ on the linear regression problem, machine learning has developed substantially. Certain model concept with great successes includes regression-classification model, Bayesian modelling, generative learning model, support vector machine, Gaussian processes, and more. Of all such, the more formal and complex model architecture created, is the concept of a *neural network*. With increasingly sophisticated architecture, heuristic approach become popular, the fast-paced advancement of the field comes with new method, new results, new observations, and its far-reaching application which led to even bigger and larger scale deployment, there has been questions about the formation and status of a theoretical ground, a rigorous matter on the side of **theoretical machine learning**.

While rigorous and well-formulated in a sense, classical and theoretical machine learning was dwarfed by the modern advancement of machine learning as a whole, leading to several anecdotal problems regarding the interpretation of phenomena, the re-evaluation of the theory to fit the more updated analysis, and explanation to more sophisticatedly designed system. This and many more, plus as present, many of such advancements and improvements are heuristic, and the general theory and conceptual understanding remain limited, led to the choice to often opt for analogies and empirical workaround. Ultimately, this resulted in multiple ‘failures’ in explaining, interpreting, and predicting behaviours of new phenomena appeared in the modern landscape of machine learning. Thereby, we suggest some particular insights and analysis, and a fix within interpretation of the phenomena.

2 OUTLINE

The main contributions of the paper lie in the heart of the problem of *machine learning theoretical*, and specifically *double descent*. As such, we focus on the following:

1. Investigation on polynomial neural network formalism as an introductory investigative result, providing reasonable insight into the process.
2. Investigation into **support vector machine** (SVM) on the binary classification landscape, or of its generality, and the data-dependent perspective.
3. Investigation into neural network architectures and their double descent exhibitions, and formalise to a component-based neural network formalism.

3 RELATED WORK

On itself, the phenomena *double descent* has been investigated somewhat comprehensively, firstly introduced by Belkin et al. (2019), which gives it distinctive saddle escape illustration. Further analysis was made by several literatures, particularly attributed the existence of double descent to the concept of model complexity and inductive bias, and potential explanations of such. Nakkiran et al. (2019) expanded the phenomena into deep neural network models. Their conclusion is reached by considering the perturbation of a learning procedure $\mathcal{T}$ on the effective model complexity $\text{EMC}_{\mathcal{D},\epsilon}(\mathcal{T})$ defined in the paper, separating the eventual phenomena into regions of observations, thereby in one way or another, predicting the tendency of double descent. This is also the first, arguably, "concise" definition of double descent, even though its nature is an empirical definition, including the notion of model complexity. Recently, there is also the Neural Tangent Kernel (Jacot et al. (2018)) which can be used to explain double descent, however further researches are still required. Preliminary works are done by Lafon & Thomas (2024), Schaeffer et al. (2023), and Liu & Flanigan (2023) on the role of optimization in double descent. Davies et al. (2023) attempted to unifying grokking with double descent, theorized a possibility of similarity during the generalization phase transition of the inference period, and Olmin & Lindsten (2024) attempted to explain epoch-wise double descent within the model of two-layer linear network. However, it is mentioned that it can also be expanded into deep nonlinear networks. Further literatures on double descent varies, but fairly new, ever since their discovery articles in Belkin et al. (2019) and Nakkiran et al. (2019). Since then, we have Shi et al. (2024); Schaeffer et al. (2023); Quéty & Tartaglione (2023a); Lafon & Thomas (2024); Neal (2019); Davies et al. (2023); Quéty & Tartaglione (2023b); Liu & Flanigan (2023); Mei & Montanari (2020; 2019a); Adlam & Pennington (2020); Transtrum et al. (2025); Liu et al. (2021); Allerbo (2025); Pezeshki et al. (2021); Brellmann et al. (2024); Nakkiran (2019b); Mei & Montanari (2019b); Heckel & Yilmaz (2020); Nakkiran et al. (2020); Belkin et al. (2020); Yang & Suzuki (2024); Spiess et al. (2023); Nakkiran et al. (2021); Zhang (2024); Cherkassky & Lee (2024); Nakkiran (2019a). An even more exotic version of double descent, dubbed *triple descent* (or more), is discovered in d' Ascoli et al. (2020).

4 PRELIMINARIES

This section introduces and provides usable preliminary that will be used throughout this text, which include learning theory, practical consideration, algorithmic guarantees, and so on.

4.1 CLASSICAL PROBLEM SETTING

Most of the paper will argue about different features of statistical learning theory (Sterkenburg (2024); Vapnik (1999); Hajek & Raginsky (2021)). It is then imperative to discuss the background setting of such. We are given the observations, or dataset of the form $\mathcal{S} = (\mathcal{X}, \mathcal{Y}) \subset \mathbb{R}^n \times \mathbb{R}^m$, of all 2-tuple pairs, assumed to be sampled or observed and governed by a distribution $\mathcal{D}$.

$$\mathcal{S} = \{(x_1, y_1), (x_2, y_2), \dots (x_n, y_n)\} \subset \mathcal{X} \times \mathcal{Y}$$

The dataset is assumed to be i.i.d. sampling according to $\mathcal{D}$, which is unknown by the priori. Hence, we have the first set of assumption.

Assumption 4.1. *The observational space is governed by a particular sampling distribution $\mathcal{D}$, for fixed y -response according to the concept $c \in \mathcal{C}$. The sampling process is further assumed to hold no further structure, hence is identically and independently distributed (i.i.d.).*

The learning problem is then formulated as followed. Given the machine learning model expressed a hypothesis h of the hypothesis class $\mathcal{H}$, the learning theory aims for creating a procedure to learn either elements of the concept class $\mathcal{C}$ of all concepts $c : \mathcal{X} \rightarrow \mathcal{Y}$, or the function class $\mathcal{F}$ of all functions $f : \mathcal{X} \rightarrow \mathcal{Y}$, which is usually set to $\{0, 1\}$ or $[0, 1]$. This distinction is trivial, hence, if it is clear, we will talk about the concept class $\mathcal{C}$ only as representative form. The learner $\mathcal{L}(h)$ consider the set of possible hypothesis $\mathcal{H}$, in which might not

coincide with $\mathcal{C}$. It receives a partial image of sample $S = (x_1, \dots, x_2, \dots, x_n)$ drawn i.i.d. according to $\mathcal{D}$ as well as the label $(c(x_1), \dots, c(x_n))$. This constitutes the dataset $\mathcal{S}$, which are based on specific concept $c \in \mathcal{C}$ for the hypothesis to learn. The task is then to use (or *extract*) meaningful information to select a hypothesis $h_S \in \mathcal{H}$ that accurately mimic c , with marginal error Θ . The notation h_S stands for all hypothesis that can be inferred from the range of the dataset (the first argument, $\mathcal{X}$).

The marginal error Θ is considered of two parameters, the empirical error $\hat{R}(h)$ and the generalization error $R(h)$. We give the following definition of them.

Definition 4.1 (Empirical risk). *Given a hypothesis $h \in \mathcal{H}$, a target concept $c \in \mathcal{C}$, and a sample $S = (x_1, \dots, x_m)$. For some particular $\epsilon > 0$, the **empirical error** or **empirical risk** of h is defined by*

$$\hat{R}_S(h) = \mathbb{P}_{x \in S \sim \mathcal{D}}[\ell\{h(x), c(x)\} \geq \epsilon] = \frac{1}{m} \sum_{i=1}^m \ell\{h(x_i), c(x_i)\} \quad (1)$$

Definition 4.2 (Generalization risk). *Given a hypothesis $h \in \mathcal{H}$, a target concept $c \in \mathcal{C}$, and an underlying distribution $\mathcal{D}$ on $\mathcal{X}$. For some particular $\epsilon > 0$, the **generalization error** or **risk** of h is defined by*

$$R(h) = \mathbb{P}_{x \sim \mathcal{D}}[\ell\{h(x), c(x)\} \geq \epsilon] = \mathbb{E}_{x \sim \mathcal{D}}[\ell\{h(x), c(x)\}] = \int_{x \in \mathcal{D}} \ell\{h(x), c(x)\} dP(x) \quad (2)$$

In essence, the empirical risk measure the hypothesis-to-observation error, and the generalization risk captures the difference of the hypothesis to the actual concept. Notice that by doing this, we have implicitly assumed that the observations and the concept is not the same thing, under the majority of scenarios. For the observations to be *approximately equal* or sufficiently captures the concept, there then would have to be various criteria to fulfil. This can be illustrated as Illustration 1.

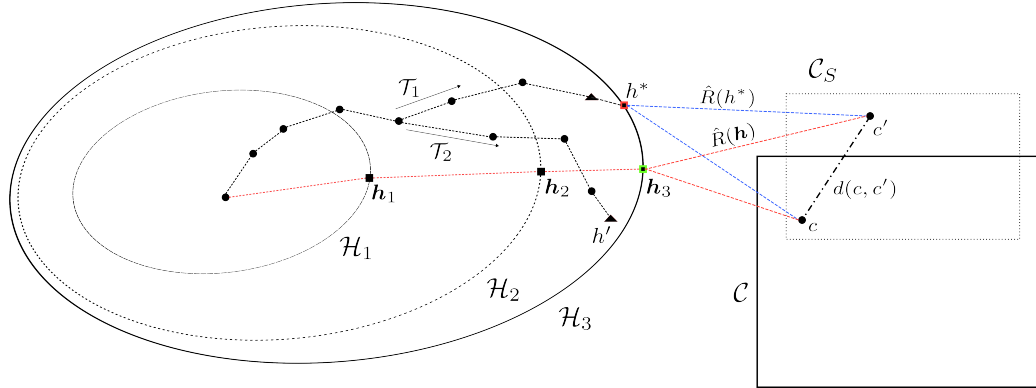


Figure 1: Illustrative dynamic of the learning problem. For an incremental hypothesis space sequence $\mathcal{H}_1, \mathcal{H}_2, \mathcal{H}_3$, we aim to obtain the procedure $\mathcal{T}$ that would either reach the *generalization solution* h , or the *empirical solution* h^* for a particular hypothesis class, with respect to the concept c and its observed concept c' . Model selection hence dictates how an algorithm or procedure than choose the best possible hypothesis to approximate the generalization solution from the empirical solution set. Do notice that here we explicitly state that the data would create a **proxy concept** that overlaps with the concept class and of some arbitrary ‘distance’ from the true concept by $d(c, c')$. In case the two classes overlap, there still exists the arbitrary distance. Also, we can also observe the intuitive notion of increasing hypothesis class - while it indeed can help in getting closer to the concept class, the somewhat intrinsic property of current learning theory being the relative random procedure class make it probabilistically unstable.

For fixed $\mathcal{H}$, for fixed and sufficiently large $\mathcal{S}$, and no observation (data) errors, the empirical risk is the generalization risk. These two measures between h and c constitute the learning problem, which can also be separated into both cases - either empirical learning or generalization learning, one to minimize $\hat{R}(h)$, and one to minimize $R(h)$.

Definition 4.3 (Empirical learning problem). *We present the formal form of the empirical learning. Suppose we have a **target**, $c \in \mathcal{C}$, where $\mathcal{C}$ is an arbitrary concept class that captures targets of the same type. Suppose*

we are provided a set of observations $\mathcal{S}$. The problem is to use certain algorithm $\mathcal{A}$ using $\mathcal{S}$, to obtain a hypothesis h^* for a fixed $\mathcal{H}$ such that:

$$R(h^*) = \min_{h \in \mathcal{H}} \hat{R}(h) = \min_{h \in \mathcal{H}} \mathbb{E}_{x \sim \mathcal{D}, x \in \mathcal{S}} \ell\{h(x), c(x)\} \quad (3)$$

The generalization best h is also called in literature as the *Bayes model*, such that it reduces the Bayes risk that is the infimum of the generalization risk, $R^* = \inf_{h \in \mathcal{H}} R(h)$. The hypothesis h^* is often called the *empirical best*, for it being the minimal, finite hypothesis of the lowest loss evaluation on the entire observation space $\mathcal{S}$. There exists no certified assumption regarding whether h^* aligns with the minimal generalization error.

Definition 4.4 (Generalization learning problem). *We present the formal form of the generalization learning problem. Suppose we have a **target**, $c \in \mathcal{C}$, where $\mathcal{C}$ is an arbitrary concept class that captures targets of the same type. Suppose we are provided a set of observations $\mathcal{S}$. Supposed we have an algorithm $\mathcal{A}$ that for fixed hypothesis space $\mathcal{H}$, equation 3 holds true. The problem is to use certain algorithm $\mathcal{A}'$ such that, under limited availability, to obtain $\mathbf{h}$, satisfies:*

$$R(\mathbf{h}) = \min_{h \in \mathcal{H}} R(h) \leq \{\epsilon\}, \quad \epsilon > 0 \quad (4)$$

For a set of risk bounds ϵ . If the setting is deterministic, then there exists $\epsilon = 0$ accordingly.

Then, we can partially say that generalization problem is an advancement from empirical solution. As such, empirical solution imposes the observed concept, and not the concept itself. This is reflected in modern machine learning landscape, by modelling the concept as distributions, and then using notions such as KL-divergence to measure such relative disparity.

Optimization problem 1: Empirical-generalization learning dynamics (non-proxy)

Data: $(x_i, y_i) \in \mathcal{S}, \Gamma \supset \mathcal{S}$, objective $\ell(y, f(x)), \nabla(\cdot)$ differentiable gradient field on ℓ .

Result: Expected optimized model to the minimal error on the objective $\ell(\cdot, \cdot)$, dependent on $\mathcal{S}$.

begin

$\Gamma \supset \mathcal{S}$, the general set;

$R(\cdot), \hat{R}(\cdot) \leftarrow \ell$ as objective functional;

for Fixed $\mathcal{H}$, with $h \in \mathcal{H}$, for $\epsilon > 0$ **do**

Find $h \rightarrow h'$, using algorithm $\mathcal{A}$ for minimization objective, such that

$$\mathcal{A} \left(\min_{h \in \mathcal{H}} \hat{R}_{\mathcal{S}}(h) + R(h) \right) \leq \epsilon \quad (5)$$

for procedure $\mathcal{A}(\cdot, \cdot)$ of structural constraints.;

end

end

return Optimized model h on arbitrary objective space (ℓ, ∇) .

These two stages of learning procedure comes of as the intrinsic design of machine learning setting. As opposed to interpolation, where empirical learning problem would have to be put into first priority to achieve the best possible point-through model, machine learning leans on approximation more and a separated criterion that is only visible in the learning setting - the fact that we are using c' to extract c . This disparity is exactly the **generalization problem**, of which is then also the goal of machine learning, to approximate the general solution using the observed solution. One then can ask what is the difference between the two notions, and when we can guarantee that both will 'converge' to the same point. The process of **model selection** is then the procedure that will optimize $h \in \mathcal{H}$ to a given target fixture, such as the empirical best.

One of the major assumption or observation from the point of the hypothesis class, is that the hypothesis cannot know the generalization error. Thereby, the more efficient and often used procedure/algorithm in training, is then the *empirical risk minimization* (ERM) method, minimizing only the empirical risk to the empirical best; then, certain measures to 'generalize' this heuristically is apprehended on the hypothesis. The process of model selection is also the place that we get the concept of **bias-variance tradeoff**.

4.1.1 UNIFORM CONVERGENCE

Suppose that for a concept class $\mathcal{H}$, we can then partition it to $\mathcal{H} = \bigcup_{k \in \mathbb{N}} \mathcal{H}_k$ for finite k hypotheses. Assume that they satisfy the uniform convergence (Shalev-Shwartz et al. (2010)):

$$\sup_{\mathcal{D}} \mathbb{E}_{S \sim \mathcal{D}^m} \left[\sup_{h \in \mathcal{H}} [R(h) - R_S(h)] \right] \xrightarrow{m \rightarrow \infty} 0 \quad (6)$$

This uniform convergence holds for a learning problem, if the empirical risks of hypotheses in the hypothesis class converges to their population risk uniformly, with a distribution-independent rate. The result of this is that a problem then can be considered learnable with the ERM rule, for any given characterization. We state the following definition.¹

Definition 4.5. A learning problem is learnable if there exist a learning rule $\mathcal{A}$ and a monotonically decreasing sequence $\epsilon_{cons}(m)$ such that $\epsilon_{cons}(m) \xrightarrow{m \rightarrow \infty} 0$ and

$$\forall \mathcal{D}, \quad \mathbb{E}_{S \sim \mathcal{D}^m} [F(\mathcal{A}(S)) - F^*] \leq \epsilon_{cons}(m), \quad F^* = \inf_{h \in \mathcal{H}} R(h) \quad (7)$$

A learning rule $\mathcal{A}$ for which this hold is denoted as a *universally consistent learning rule*.

There are certainly a fundamental class of rule or algorithm $\mathcal{A}$ to resolve this particular uniform convergence of classes. One of the major assumption or observation from the point of the hypothesis class, is that the hypothesis cannot know the generalization error. Thereby, one strategy is the *empirical risk minimization* (ERM) method, minimizing only the empirical risk to the empirical best; then, certain measures to ‘generalize’ this heuristically is apprehended on the hypothesis.

Definition 4.6. A rule $\mathcal{A}$ is an ERM (Empirical Risk Minimizer), denoted $\mathcal{A}_{ERM}$ if it minimizes the empirical risk $\hat{R}(h) = (1/m) \sum_{c \in \mathcal{C}} \ell(h, c)$, over the observational space S ,

$$\hat{R}_S(\mathcal{A}_{ERM}(S)) = \hat{R}_S(h_S) = \inf_{h \in \mathcal{H}} \hat{R}_S(h) \quad (8)$$

Hence, the ERM tries to obtain $\text{ERM}(h') = \arg \min_{h \in \mathcal{H}} \hat{R}(h)$.

The problem of overfitting still somewhat resides in the setting of the method. Hence, usually, we have an implicit term in addition, called the *regularizer*, of which is as followed.

Definition 4.7. A rule $\mathcal{A}$ is an SRM (Structural Risk Minimizer), denoted $\mathcal{A}_{SRM}$ if it minimizes the empirical risk $\hat{R}(h) = (1/m) \sum_{c \in \mathcal{C}} \ell(h, c)$, over the observational space S ,

$$\hat{R}_S(\mathcal{A}_{SRM}(S)) = \hat{R}_S(h_S) + \lambda r(h) = \inf_{h \in \mathcal{H}} \hat{R}_S(h) + \lambda r(h) \quad (9)$$

of the regularizing term $r(\cdot)$. Hence, the SRM tries to obtain $\text{ERM}(h') = \arg \min_{h \in \mathcal{H}} \hat{R}(h)$, while taking into consideration a forcing term r , usually defined on the model complexity.

In this paper, we would like to presume the ERM method in question as the famous *gradient descent*. We refer to Achlioptas; Ruder (2017) for general view of the algorithm, and Zhang (2019) for a source on particularly gradient descent algorithm on deep learning models.

4.2 OVERFITTING AND UNDERFITTING

Following the above framework of statistical probabilistic learning theory, one understanding about the concept of underfitting and overfitting (similar to James et al. (2013)) can be reached, using the notion of *partitioning* of observation space. Such work is typically done because of **practical condition**, in which the following constraints is applied.

¹The reason why $r(\cdot)$ is not characterized in this preliminary section, is because of their uncertainty of effect. Hypothetically, $r(\cdot)$ is an additional structure (bias) of which apply a new gradient field in conjunction to the loss gradient field itself. This addition is uncertain of its content and analysis, since it depends on various stochastic dynamic of the model. We reserve from using regularization for now because of such reason.

Assumption 4.2. *No knowledge of $c \in \mathcal{C}$ is provided.*^a

^aFew points need to be said about this assumption. This assumption represents the *worst-case possible* of a system dynamic. From the perspective of a designer, inherent leakage of information can be provided in the design of a model. In such, case we are operating in the mode of a *resource-constrained, closed* game, in which the model with further assumptions of fixed class, operates on. The problem space itself is inherently large and penalize any additional structural changes. If one is to occur, we usually mention it as *implicit bias*, but another name that would be more fitting would be *structural attractor*, after the notion of gradient attractor in chaotic systems.

Without such knowledge, $R(h)$ cannot be computed, accordingly. We specify such approach as *heuristic*, as it is only an approximation, and for better definition, we define it as followed.

Definition 4.8 (Heuristic empirical risk). *Given a dataset S of size m , supposedly of the concept $c \in \mathcal{C}$, and the hypothesis $h \in \mathcal{H}$, then the **heuristic empirical risk** take a partitioning $k < m$ with $k/m \geq 1/2$, hence defining the heuristic empirical loss on k , denoted $\hat{R}_k(h)$.*

Definition 4.9 (Heuristical generalization risk). *Given a dataset S of size m , supposedly of the concept $c \in \mathcal{C}$, and the hypothesis $h \in \mathcal{H}$. Fixed $\hat{R}_k(h)$, for the final optimization of algorithm $\mathcal{A}$, then the **heuristic generalization risk** take the remaining 2-partition of portion $p = k - m$, hence defining the heuristic generalization loss on p , denoted $\hat{R}_p(h_{\mathcal{A}})$.*

The two heuristic definition however, can be noted to not be equal to the true generalization risk - empirical risk pair in reality. Nevertheless, one can then define overfitting and underfitting in such manner.

Definition 4.10 (Underfitting). *A model h is called **underfitting** if for a constant $\epsilon > 0$ chosen for the setting, then $\hat{R}_k(h_{\mathcal{A}}) > \epsilon$, $\hat{R}_p(h_{\alpha}) > \epsilon$ over the entire observation space.*

Usually, underfitting is defined to be such that test error is smaller than training error partition, however, we do not think that is a good definition. Especially since, at least in this setting, the empirical error under algorithm $\mathcal{A}$ is reduced to the representative smallest error that $\mathcal{A}$ can achieve on that particular model, within particular setting of h , then come the heuristic generalization error. Considering also that $k/m \geq 1/2$, that is, at least the training partition is half of the observation set, then only under very specific case will $\hat{R}_p(h_{\alpha})$ be smaller than $\hat{R}_k(h)$, statistically speaking. Thence this line of thought also reinforces the idea that both of them will be larger than the threshold ϵ . Usually, heuristically, this threshold of error is 0.3, normalized. Next, we define the notion of overfitting.

Definition 4.11 (Overfitting). *A model h is called **overfitting** if there exists a specific constant $\epsilon > 0$ chosen for the setting, such that $\hat{R}_k(h) < \epsilon$, $\hat{R}_p(h_{\alpha}) > \epsilon$ over the entire observation space.*

Namely, this mean that the static learning makes the generalization error worse. This draws the analogy of the student - studying explicitly for the past paper tests, yet unaware of the larger margin of questions in the large set of the test paper pool, leading to subpar performance. Similarly, overfitting is theorized to be fitting too fit to the reduced observation space - hence when it comes to the generalization observation space, it fails within such heuristic. The notion works on the mask of the risks, $\hat{R}_k$ and $\hat{R}_p(h_{\alpha})$.

4.2.1 PRACTICAL PROXY

In practice, Assumption 4.2 means we cannot have $c \in \mathcal{C}$, or its probabilistic interpretation $\mathcal{D}$ such that $x \sim \mathcal{D}$, or in the extreme non-structured case, $(x, y) \sim \mathcal{D}$. In such case, the typical practical fix relies on the partitioning of the available partitioning space, that is, the empirical-generalization proxy. For large enough singleton count of $|\{(X_k, Y_k)\}|$, we assume that $\hat{R}_{\text{Gen}}(h) \approx R(h)$. Thus, the algorithm can be stated in a form as followed.

The assumption above of the generalization set to the generalization truth error is apparent in the usual system theorem, Theorem 4.3.

Optimization problem 2: Empirical-generalization learning dynamics (proxy)**Data:** $(x_i, y_i) \in \mathcal{S}, y_i = \{\pm 1\}$, hinge objective $\ell(y, f(x))$, $\nabla(\cdot)$ differentiable gradient field on ℓ .**Result:** Expected optimized model to the minimal error on the objective $\ell(\cdot, \cdot)$, dependent on $\mathcal{S}$.**begin** $((X_1, Y_1), \dots)_n \leftarrow \text{Partition}_n(\mathcal{S});$ $\{(X_j, Y_j)\} \leftarrow \text{Train};$ $\{(X_k, Y_k)\} \leftarrow \text{Gen for } k \neq j;$ **for** Fixed $\mathcal{H}$, with $h \in \mathcal{H}$, for $\epsilon > 0$ **do**(Optimistically) achieve $h \rightarrow h'$, using algorithm $\mathcal{A}$ for minimization objective, such that

$$\mathcal{A} \left(\min_{h \in \mathcal{H}} \hat{R}_{\text{Train}}(h) + \hat{R}_{\text{Gen}}(h) \right) \leq \epsilon \quad (10)$$

for procedure $\mathcal{A}(\cdot, \cdot)$ of structural constraints.;**end****end****return** Optimized model $h_{\mathcal{A}}$ on arbitrary objective space (ℓ, ∇) .**Theorem 4.3** (Mohri et al. (2012)). *For fixed $\mathcal{H}$, for fixed and sufficiently large $\mathcal{S}$, and no observation errors, the empirical risk is the generalization risk:*

$$\mathbb{E}_{\mathcal{S} \sim \mathcal{D}^m} [\hat{R}_{\mathcal{S}}(h)] = R(h) \quad (11)$$

Proof. By linearity of the expectation, and the fact that we assume the sample is given i.i.d., we can write:

$$\mathbb{E}_{\mathcal{S} \sim \mathcal{D}^m} [\hat{R}_{\mathcal{S}}(h)] = \frac{1}{m} \sum_{i=1}^m \mathbb{E}_{\mathcal{S} \sim \mathcal{D}^m} [\mathbb{1}_{h(x_i) \neq c(x_i)}] = \frac{1}{m} \sum_{i=1}^m \mathbb{E}_{\mathcal{S} \sim \mathcal{D}^m} [\mathbb{1}_{h(x) \neq c(x)}],$$

for any x in sample $\mathcal{S}$. Thus,

$$\mathbb{E}_{\mathcal{S} \sim \mathcal{D}^m} [\hat{R}_{\mathcal{S}}(h)] = \mathbb{E}_{\mathcal{S} \sim \mathcal{D}^m} [\mathbb{1}_{h(x) \neq c(x)}] = \mathbb{E}_{x \sim \mathcal{D}} [\mathbb{1}_{h(x) \neq c(x)}] = R(h).$$

which yields the desired result. $\square$

5 PROBLEM STATEMENTS

Before continuing further, we must review the problem statement that led to the problem of double descent. We will have a look at the theory, the original conflict, and the illustrative phenomenon in action. In general, the learning setting in consideration of statistical learning theory is often supervised learning, for guarantee of a plausible upper bound result.

5.1 BIAS-VARIANCE TRADEOFF

In classical sources, James et al. (2013); Goodfellow et al. (2016); Hajek & Raginsky (2021); Mohri et al. (2012); Shalev-Shwartz & Ben-David (2014), bias-variance tradeoff is a classical principle used in the categorization of technique called *model selection*. Specifically, this refers to the progress of choosing the best predictor h^* that minimize the generalization error that is the main goal of a particular machine learning model, during the process of training and inference. Originally, the theory that bias and variance often come in to conjunction is **Estimation theory**, of which there exists an estimator $\hat{\theta}$ that use a set of observations, to estimate the concept, or the underlying mechanics of certain system that outputted the observation set, by the **probabilistic perspective** - that is, it is governed by appearance by an independently and identically distributed process of parameterized probability $\theta \in \Theta$. Here, parameterized means being expressed by a set, often finite, of parameters, by E. L. Lehmann (1998); Paninski (2005). It is only when Geman et al. (1992) introduced it to machine learning that we have an equivalent structure that is similar to bias-variance in machine learning.

5.1.1 PRECURSOR (GEMAN ET AL., 1992)

The original definition of bias-variance tradeoff by Geman et al. (1992) is first constructed using the mean-square error, which is regarded as a normal measure in the real encoding space. Their approach is to justify bias-variance via decomposition of the loss function ℓ , for such to find an alternative reasonable form of such loss landscape. Suppose of a regression problem to construct a hypothesis function $f(x)$

from $(x_1, y_1, \dots, x_N, y_N)$ for the purpose of generalization - that is, predicting unseen variational values for different pair $(x_j, ?)$ such that $? = y_j + \epsilon$ for a conceivable implicit error. To be explicit about the relation of this problem, or f on the given data $\mathcal{D} = \{(x_i, y_i) \mid i \leq N\}$, denote $f(x; \mathcal{D})$ instead of f , the natural mean-square measure as a predictor is:

$$\mathcal{M}(f, y) = \mathbb{E} [((y - f(x; \mathcal{D})))^2 \mid x, \mathcal{D}] \quad (12)$$

for $\mathbb{E}[\cdot]$ the expectation wrt to a distribution P . Decomposing the right-hand side, we have:

$$\mathcal{M}(f, y) = \mathbb{E} [((y - f(x; \mathcal{D})))^2 \mid x, \mathcal{D}] = \mathbb{E} [(y - \mathbb{E}[y \mid x])^2 \mid x, \mathcal{D}] + (f(x; \mathcal{D}) - \mathbb{E}[y \mid x])^2 \quad (13)$$

Here, $\mathbb{E} [(y - \mathbb{E}[y \mid x])^2 \mid x, \mathcal{D}]$ does not depend on $\mathcal{D}$, but simply the statistical variance of y given x . The term $(f(x; \mathcal{D}) - \mathbb{E}[y \mid x])^2$ is considered a natural measure of effectiveness on $\mathbb{R}^n$ as a singular predictor of y . Now, for $\mathbb{E}_{\mathcal{D}} [(f(x; \mathcal{D}) - \mathbb{E}[y \mid x])^2]$ which depends on the training set $\mathcal{D}$ in its computation, is decomposed into the form of *bias-variance decomposition* terms, by derivation:

$$\mathbb{E}_{\mathcal{D}} [(f(x; \mathcal{D}) - \mathbb{E}[y \mid x])^2] = \underbrace{\{\mathbb{E}_{\mathcal{D}}[f(x; \mathcal{D})] - \mathbb{E}[y \mid x]\}^2}_{\text{bias term}} + \underbrace{\mathbb{E}_{\mathcal{D}} \{(f(x; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[f(x; \mathcal{D})])^2\}}_{\text{variance term}} \quad (14)$$

We summarize this in the following statement.

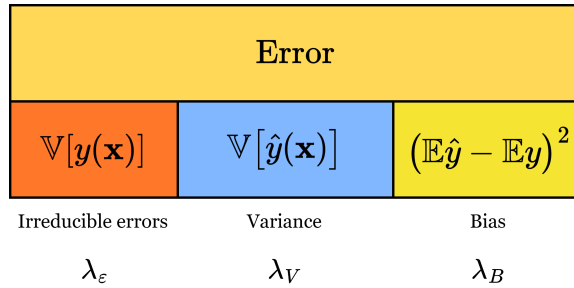
Theorem 5.1 (Bias-variance decomposition). *Suppose the model $f(x; \mathcal{D})$ for the data $\mathcal{D} = (x_i, y_i)$ and its parameter x is defined. For y_i of the target concept's responses y , and consider a regression problem with the loss measure $\mathcal{M}(f, y)$ of mean squared risk, the following statement is true:*

$$\mathbb{E}[\mathcal{M}(f, y)] = \mathcal{B}(f, y) + \mathcal{V}(f, y) + \mathbb{E}[\mathbb{E} [((y - f(x; \mathcal{D})))^2 \mid x, \mathcal{D}]] \quad (15)$$

for $\mathbb{E}[\cdot \mid x, \mathcal{D}]$ any expression with dependencies on x and $\mathcal{D}$. The bias and variance term is subsequently expressed by

$$\mathcal{B}(f, y) = \underbrace{\{\mathbb{E}_{\mathcal{D}}[f(x; \mathcal{D})] - \mathbb{E}[y \mid x]\}^2}_{\text{bias}}, \quad \mathcal{V}(f, y) = \underbrace{\mathbb{E}_{\mathcal{D}} \{(f(x; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[f(x; \mathcal{D})])^2\}}_{\text{variance}} \quad (16)$$

Figure 2: Decomposition of the error term into 3 parts. Respectively, the irreducible error $\mathbb{V}[y(\mathbf{x})]$, the variance (blue) and the bias (dark yellow). All of them are assumed to take up 100% of the error observed in such composition. The proportion is included using the three coefficient $\lambda_\epsilon, \lambda_V, \lambda_B$ where $\lambda_\epsilon + \lambda_V + \lambda_B = 1$.



The above decomposition principle is often expressed into a form where there exists the intrinsic noise Brown & Ali (2024):

$$\begin{aligned} \mathbb{E}_D \left[\mathbb{E}_{xy} (y - \hat{f}(x))^2 \right] &= \mathbb{E}_x \left[(y^* - \mathbb{E}_D[\hat{f}(x)])^2 \right] + \mathbb{E}_x \left[\mathbb{E}_D (\hat{f}(x) - \mathbb{E}_D[\hat{f}(x)])^2 \right] \\ &\quad + \mathbb{E}_{xy} [(y - y^*)^2] \end{aligned} \quad (17)$$

In all, almost all representation of bias-variance decomposition are the same. For simplicity of notation, we adopt the similar form in the standard case of the paper Adlam & Pennington (2020):

$$\mathbb{E} [\hat{y}(\mathbf{x}) - y(\mathbf{x})]^2 = (\mathbb{E}\hat{y}(\mathbf{x}) - \mathbb{E}y(\mathbf{x}))^2 + \mathbb{V}[\hat{y}(\mathbf{x})] + \mathbb{V}[y(\mathbf{x})] \quad (18)$$

A main common theme of criticism toward bias-variance tradeoff is the fact that the decomposition is much more general, and intrinsic for the class of *mean squared loss*. However, when considering the naturalness of an error measure and then its direct applicant, mean square error on real space of sufficient support and measure comes of as a very natural choice of an error measure, at least in consideration of the regression setting; for that, we then can also define classification as another top-layer above a regression's similar real continuous space, i.e. a decision layer. Furthermore, it can also be shown Brown & Ali (2024); Pfau (2013) that it also holds for the class of Bregman divergence measure.

What does the decomposition say about model selection principle in general? In general, bias-variance is typically presented of its decomposition only, and the asymptotic prediction of its behaviours. In fact, one of the reason that it became the rule-of-thumb for ML practitioner, as well as generally statistical learning (Lafon & Thomas (2024) provides a quite rigorous treatment of bias-variance tradeoff in the section on statistical learning theory) solidify the trade-off as a particular model selection principle. Generally, this tradeoff can be summarized as followed:

Theorem 5.2 (Bias-variance tradeoff). *For the expected loss of any given hypothesis h , the bias $\mathcal{B}(f, y)$ and variance $\mathcal{V}(f, y)$ is inversely proportional, that is, $\mathcal{B}(f, y) \propto \lambda^{-1} \mathcal{V}(f, y)$ for some proportionality λ that may or may not be constant. In the most general case possible, $\lambda = -1$ on the entire error range.*

The tradeoff is then of inverse proportionality. Indeed, statistically, we have such tradeoff on a statistical framework in a more concrete sense. For the bias to increase, variance will increase, of which the criterion is inverse - we would like to have more bias but lower variance, and vice versa, according to such theory.

5.1.2 EXPANSION AND VARIATIONS

The bias-variance decomposition and tradeoff is not general. Indeed, works, by Geman et al. (1992); Sharma & Aiken (2014); Domingos (2000c); Adlam & Pennington (2020); Yang et al. (2020) and more all attempted to reconstruct and reformulate bias-variance, and it did leave out a picture of uncertainty regarding the concept. Specifically, there are certainly different way to do the bias-variance decomposition, as classified in Adlam & Pennington (2020) paper. Some of this are particularly specialized in *neural network* training, basing on the main structure of modern machine learning in neural network formalism.

1. **Semiclassical approach:** The semiclassical approach is fairly simple. We can rewrite to another variation of the classical approach to be

$$\mathbb{E}[\ell(h)] = \mathbb{E}_x \mathbb{E}_\epsilon [\hat{y}(x) - y(x)]^2 = \mathbb{E}_x (\mathbb{E}_\epsilon [\hat{y}(x)] - y(x))^2 + \mathbb{E}_x \mathbb{V}_\epsilon [\hat{y}(x)] \quad (19)$$

and instead, consider the *structural conformation* of the decomposition, or rather, taking the expectation within $\theta \in \Theta$ of the structural description of $h \in \mathcal{H}$. Then, we shall have

$$\mathbb{E}_{\text{SC}}[\ell(h)] = \mathbb{E}_x \mathbb{E}_\theta \mathbb{E}_\epsilon [\hat{y}(x) - y(x)]^2 = \mathbb{E}_x \mathbb{E}_\theta (\mathbb{E}_\epsilon [\hat{y}(x)] - y(x))^2 + \mathbb{E}_x \mathbb{E}_\theta \mathbb{V}_\epsilon [\hat{y}(x)] \quad (20)$$

Do note that the order matters, since the expectation can be constant of a given ordering. And expected value over constant is just the constant.

2. **Symmetric decomposition:** This approach is fairly difficult, we refer to the paper itself of its own justification on the decomposition. The symmetric decomposition relies on the shape of the input, $\mathcal{X} = \{P, D, \epsilon\}$, of which you can remove the ϵ term and retain some decomposition on P and D per distribution.

5.1.3 GENERALIZED DECOMPOSITION

According to Domingos (2000b;a), the bias-variance decomposition can usually be treated as being controlled by three factor error, with their respective coefficients $\lambda = (\lambda_1, \lambda_2, \lambda_3)$ for such:

$$\begin{aligned} \mathbb{E}_{\mathcal{D}} [d((f, \mathbb{E}[y | x]))] &= \lambda_1 \text{Bias}(f, y) + \lambda_2 \text{Var}(f, y) + \lambda_3 \epsilon(\mathcal{D}) \\ &= \lambda_1 \underbrace{\{\mathbb{E}_{\mathcal{D}}[f(x; \mathcal{D})], \mathbb{E}[y | x]\}}_{\text{bias term}} + \lambda_2 \underbrace{\mathbb{E}_{\mathcal{D}} \{(f(x; \mathcal{D}), \mathbb{E}_{\mathcal{D}}[f(x; \mathcal{D})])\}}_{\text{variance term}} + \underbrace{\lambda_3 \epsilon}_{\text{irreducible error}} \end{aligned}$$

where $\epsilon(\mathcal{D})$ is the irreducible error of the system, depends (theoretically) on the intrinsic imperfection of the dataset $\mathcal{D}$. This method of which the bias-variance-irreducible error are 'fit' into the total error by three coefficients $\lambda_1, \lambda_2, \lambda_3$ instead. This does not only scale up the B-V-IE triplet, but also leaves out the inexplorable gap between the scaling, and the actual normal measure.

5.2 DOUBLE DESCENT

Nevertheless, of bias-variance and the analogous statistical learning theory concept, the target is the same. It is the dilemma of which is presented in **Occam's razor**, for choosing the sufficient model of good complexity, or bias, for tradeoff of its generalization ability, or variance. Then there must exist a sweet spot between the axis of bias and variance, since they are as exhibited above inversely proportional to each other. However, double descent seemingly broke the status quo, and insists on an interesting phenomenon - under the same setting, if we 'crank' the complexity high enough, we will then reach a point then called the **interpolation threshold**, such that the trend reverse and the error rate, instead of being theorized to go up, goes down to a certain line of lower bound.

The first identification of the double descent phenomena dated back to the paper of Belkin - Belkin et al. (2019), in which the title is literally "reconciling" modern machine learning practice and the bias-variance tradeoff. In modern machine learning practice, or state-of-the-art developments, models are now bigger than ever. If to notice, we will see that currently models are inherently large, for example, a normal large language model will have from 900 millions (900M) to a few billions, for example 10 billions (10B) parameters. That is not taking into account the overall dynamics and structure of the model, which dictates the operating range and efficiency of the model itself. These model, based on the neural network architecture are somewhat trained to exactly fit (or interpolate) the data, almost certainly so that it turn from a prediction setting to an estimation setting. By statistical learning theory, this would be considered overfitting, and yet, they often obtain very high accuracy on test data.

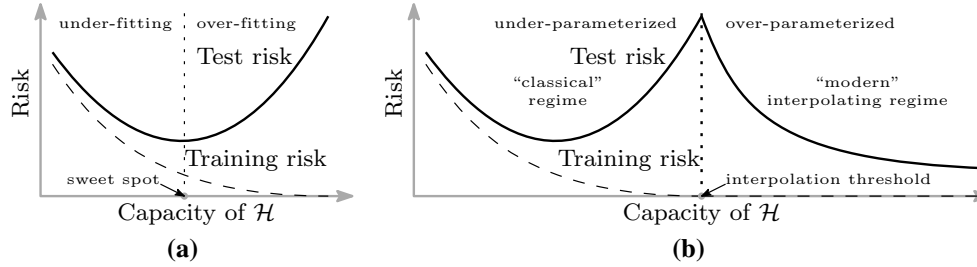


Figure 3: **Curves for training risk (dashed line) and test risk (solid line).** (a) The classical *U-shaped risk curve* arising from the bias-variance trade-off. (b) The *double descent risk curve*, which incorporates the U-shaped risk curve (i.e., the “classical” regime) together with the observed behaviour from using high capacity function classes (i.e., the “modern” interpolating regime), separated by the interpolation threshold. The predictors to the right of the interpolation threshold have zero training risk. Reproduced from Belkin et al. (2019).

The main finding that Belkin found is a pattern for how the apparent performance on unseen data depends on model capacity and the mechanism underlying the emergence of double descent. When function class capacity is below the “interpolation threshold”, learned predictors exhibit the classical *U-shaped* curve from Figure 3. The ‘modern’ interpolating regime marks the opposite trend to the right, where the risk starts to decrease up to a lower bound, which then can be called the *optimal descent bound*.

The bottom of the *U* is achieved at the sweet spot which balances the fit to the training data and the susceptibility to over-fitting: to the left of the sweet spot, predictors are under-fit, and immediately to the right, predictors are over-fit. When we increase the function class capacity high enough (e.g., by increasing the number of features or the size of the neural network architecture), the learned predictors achieve (near) perfect fits to the training data—i.e., interpolation. Although the learned predictors obtained at the interpolation threshold typically have high risk, we show that increasing the function class capacity beyond this point leads to decreasing risk, typically going below the risk achieved at the sweet spot in the “classical” regime. (Belkin et al. (2019))

At this point, we would like to also clarify the term interpolation threshold in this case. Usually, under the pretext of overfitting and underfitting, interpolation threshold means the following:

Definition 5.1 (Interpolation threshold). *Assuming a typical supervised learning setting, for a hypothesis h within class structure $\mathcal{H}$, and a supposed concept c of the class $\mathcal{C}$. Suppose $[X, Y] \subset c \in \mathcal{C}$ being the dataset, or observable space of c on which h is tested on. Under the **empirical generalization learning** procedure,*

partitioning $[X, Y]$ into $[X, Y]_1$ and $[X, Y]_2$, for error evaluated on the first set, $\hat{R}_1(h)$ called *training error*, and the second set, $\hat{R}_2(h)$ called *test error*, with optimization applied on the first set, then evaluated statically on the second, the **interpolation threshold** with respect to a particular complexity measure of h , $\mathcal{M}(h)$, is the point $\mathcal{M}_0(h)$ where it is possible for $\hat{R}_2(h)/\hat{R}_1(h) \approx 1$, for $\hat{R}_1(h) \ll \epsilon$, $\hat{R}_2(h) \ll \epsilon$, with $\epsilon \geq 0$.

This basically means that what is learnt, with respect to the model complexity that gauge *how much* and *what* the model can learn, perfectly fits the concept c itself, from the observation space on its own. We can say that, that specific sense, we gain the assumption that the observation space is *fully representative* of the concept class, and the concept class is then fully realized by the model. Such point where that is possible, or there then such behaviour is possible, is called the interpolation point. The definition relies on the statement ‘where it is possible’, hence it still, does not give us a particularly sound notion on to how and what constitute such possibility. The term empirical generalization learning comes from the fact that the procedure in practice of estimating and optimizing train-test-validation set in itself, is an imperfect empirical approximation of generalization testing. While the fact that we cannot gauge generalization error, in a purely blind black-box setting is settled (Mohri et al. (2012); Sugiyama (2015); Shalev-Shwartz & Ben-David (2014)), the method is then to try to mimic such by using generalization-via-hidden observations. To facilitate this, deliberately, the empirical learning procedure is only dynamic during training session, and static throughout the whole testing and validation testing section, and further on.

By default, in general theory, such interpolating point is dangerous. Not only because it is inherently difficult to know exactly what the dataset constitute, whether it is actually representative of the concept that gives rise to it, or the structural information, noises, and interferences in between. It is hence impossible, to a certain point, to truly know the effectiveness of the model beside interpolating the current dataset which forms the observable details.

Ever since Belkin et al. (2019), despite the general chaotic view of double descent that seems to hold up pretty well, many researches have been conducted throughout, trying to observe or explain it, as we briefly mentioned Nakkiran et al. (2019) as one of such main paper. Before attempting to do anything, and to further contribute with hypotheses forward, we will have to resolve to review previous coursework on such matter. Most of those works include some on theoretical analysis, some on dissection of optimization method then speculation of possible hindsight, some on experiments and observable effects, and so on.

As such, the first thing to do is to contrive the consensus on ‘what’ constitutes double descent, at least per observable results, by delivering a conjecture.

Conjecture 5.1. *On the landscape of double descent, model complexity versus generalization error, there exists a fundamental point $\mathcal{M}_0$ of model complexity axis, where $dR(h)/d\mathcal{M} < 0$ for $\mathcal{M} > \mathcal{M}_0$, which is the interpolation point.*

While this is not perfect, it provides us with a general idea on where to look for double descent. With this, we are ready to follow up the literatures in provision of said questions.

5.2.1 DISCOVERY OF DOUBLE DESCENT

While double descent is discovered by Belkin et al. (2019) and further on modern deep learning landscape of Nakkiran et al. (2019), it remains that the conclusion is not reached if there exists some fundamental models that would not have double descent, the consistency of double descent through different models entirely and so on. This is mainly because of a particular ‘shock effect’, since before double descent, bias-variance holds the candle as the main principle of choosing and optimizing models to its optimal saddle point. Suddenly by then we observe double descent, which introduces the interpolating threshold $\mathcal{M}_0$, that breaks this principle. There is then the open question on how far-reaching is double descent in consideration of different architectures, and this is reflected in several papers above. As of now, the main structures that have been verified includes

- **Classical models** Yang & Suzuki (2024); Nakkiran (2019b); Schaeffer et al. (2023); Lafon & Thomas (2024) (linear regression and classical polynomial), Li & Sonthalia (2024) (linear regression upon least square), Liu et al. (2021), Zhang (2024); Allerbo (2025) (kernel methods/regressions), Brellmann et al. (2024) (Classical MSBE-LSTD² optimization), Harzli et al. (2023) (Gaussian process). Some papers also have experiments on Random Fourier Feature Networks, and so on, but not substantial (including Belkin et al. (2019) paper).
- **Deep neural networks:** Luo & Dong (2023); Yang & Suzuki (2024); Pezeshki et al. (2021) (CNN, VGG, ResNet, DenseNet), Nakkiran et al. (2019) (Transformer), Shi et al. (2024) (GNN).

²Mean-Squared Bellman Error and Least-Squared Temporal Difference

Most of the experiments are concerned of seeing and analysing double descent, basic analysis and experimental observations, test case scenarios for different models and setting and so on. All of the above models are then considered to be, concretely, reported of observing double descent, through various testing scenario on them. Review papers and literatures can be found as Schaeffer et al. (2023); Hu (2021), and Shi et al. (2024) on specifically, graph neural network.

5.2.2 DEEP DOUBLE DESCENT, NAKKIRAN ET AL. (2019)

Another prominent discovery to look at is Nakkiran et al. (2019), on the double descent of deep learning models. The paper serves as the first observation upon which double descent is observed on modern, deep learning theoretic models, especially complex one like Transformer or ResNet networks.

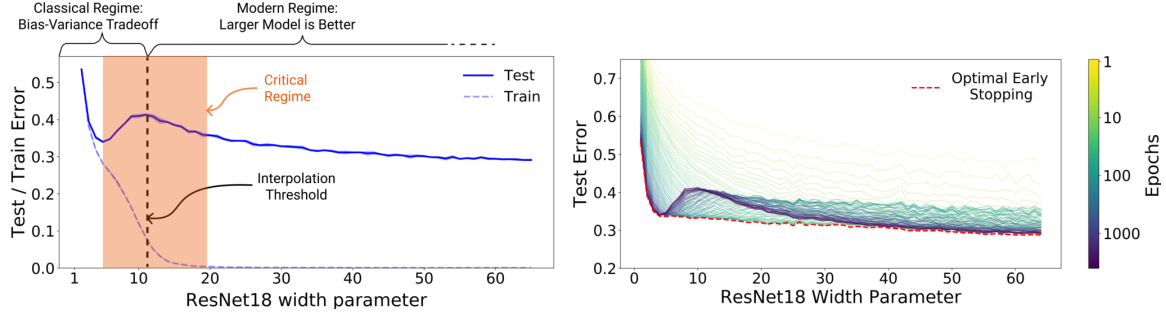


Figure 4: **Left:** Train and test error as a function of model size, for ResNet18s of varying width on CIFAR-10 with 15% label noise. **Right:** Test error, shown for varying train epochs. All models trained using Adam for 4K epochs. The largest model (width 64) corresponds to standard ResNet18. Resued from Nakkiran et al. (2019).

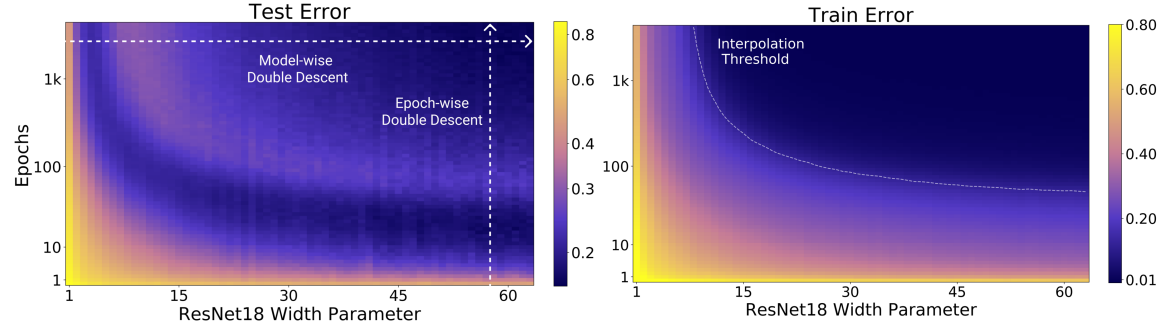


Figure 5: **Left:** Test error as a function of model size and train epochs. The horizontal line corresponds to model-wise double descent—varying model size while training for as long as possible. The vertical line corresponds to epoch-wise double descent, with test error undergoing double-descent as train time increases. **Right** Train error of the corresponding models. All models are Resnet18s trained on CIFAR-10 with 15% label noise, data-augmentation, and Adam for up to 4K epochs. Reused from Nakkiran et al. (2019)

They define *effective model complexity* of $\mathcal{T}$ (w.r.t. distribution $\mathcal{D}$) to be the maximum number of samples n on which $\mathcal{T}$ achieves on average ≈ 0 training error. This is an entirely empirical definition, similarly per definition as VC-dimension.

Definition 5.2 (Effective Model Complexity). *The Effective Model Complexity (EMC) of a training procedure $\mathcal{T}$, with respect to distribution $\mathcal{D}$ and parameter $\epsilon > 0$, is defined as:*

$$\text{EMC}_{\mathcal{D},\epsilon}(\mathcal{T}) := \max \{n \mid \mathbb{E}_{S \sim \mathcal{D}^n} [\text{Error}_S(\mathcal{T}(S))] \leq \epsilon\}$$

where $\text{Error}_S(M)$ is the mean error of model M on train samples S .

Using this definition, their main hypothesis can then be stated as the following three-fold regions:

Hypothesis 5.1 (Generalized Double Descent hypothesis, informal). *For any natural data distribution $\mathcal{D}$, neural-network-based training procedure $\mathcal{T}$, and small $\epsilon > 0$, if we consider the task of predicting labels based on n samples from $\mathcal{D}$ then:*

Under-parameterized regime. *If $\text{EMC}_{\mathcal{D},\epsilon}(\mathcal{T})$ is sufficiently smaller than n , any perturbation of $\mathcal{T}$ that increases its effective complexity will decrease the test error.*

Over-parameterized regime. *If $\text{EMC}_{\mathcal{D},\epsilon}(\mathcal{T})$ is sufficiently larger than n , any perturbation of $\mathcal{T}$ that increases its effective complexity will decrease the test error.*

Critically parameterized regime. *If $\text{EMC}_{\mathcal{D},\epsilon}(\mathcal{T}) \approx n$, then a perturbation of $\mathcal{T}$ that increases its effective complexity might decrease **or increase** the test error.*

It is to notice that this definition is also only observational. By that, it only outlines the specific region of interest where the paradigm shift is identified through experimental results. It has no effective predictive power, or rather descriptive power more than setting up hypothesis with respect to the effective model complexity, and some arbitrary perturbation. Nakkiran et al. (2019) also noticed this difficulty in providing a theoretical definition and theorem regarding such hypothesis, as said in their manuscript. The existence of the critical regime is also not defined.

The behaviour itself has been particularly investigated, in certain settings. For example, before Belkin et al. (2019), Advani & Saxe (2017) investigated the generalization error in neural networks of high-dimensional measure. Of such, various behaviours that bear similarity to double descent can be observed. Belkin et al. (2018) expanded on his previous research on the particular subset of kernel learning models, providing a theoretical analysis of specifically the Laplacian kernel on standard neural network. Mei & Montanari (2020) investigated random feature regression in such regard, and also found similar results.

The fact that double descent is not well-defined itself is a problem on its own. Up to the author's knowledge and of myself, there has been no effective definition or given description that outlines specifically how double descent can be structured. Instability in reproducing experiments and the effective range of the phenomena remains a question on its own.

6 THE POLYNOMIAL TRANSITION PROBLEM

Polynomial space is one of those famous example that is often cited of the bias-variance tradeoff, for example, in Goodfellow et al. (2016); Sugiyama (2015). Indeed, it is considered, usually, a very effective and high-volatility construct class. Let us reiterate what we know about polynomial class.

There exists two type of elementary polynomial classification, based on their (i) variable count or (ii) its domain range. We first define the univariate polynomial function.

Definition 6.1 (Univariate polynomial). *A function of a single variable t is a polynomial on its domain if we can put it in the form*

$$a_n t^n + a_{n-1} t^{n-1} + \dots + a_1 t + a_0 t^0 \quad (21)$$

where $\{a_i\}$ are constants, and of a singular variable t .

From this, we can extend to polynomial of multiple variables. In general, a function $f(x, y)$ is a *multivariate polynomial* in x, y if and only if it can be represented as a finite sum of terms of the form $c x^k y^m$, where c is a coefficient and k and m are nonnegative integers. The number $k + m$ is called the degree of the term, and the degree of the polynomial $f(x, y)$ is equal to the highest degree of its own terms. We can then generalize multivariate polynomial and univariate polynomial in the same kind.

Definition 6.2 (Polynomial, multivariate generalization). *Let K be a field. We then denote $K[x_1, \dots, x_n]$ the space of all polynomials in n variables, $x_1, \dots, x_n$. A **multivariate monomial** is then an expression $x_1^{\alpha_1} \dots x_n^{\alpha_n}$ where $\alpha_1, \dots, \alpha_n$ are integers. The degree δ of such multivariate polynomial is the largest degree of terms of any monomial term.*

Effectively, we can see that multivariate polynomial can take on many forms, and also unconventional forms. In order to capture any reasonable details, most of the time focus are on symmetric polynomials $f(x, y)$ in which $f(x, y) = f(y, x)$, or homogeneous polynomials in which all the terms are of the same degree.

Given such, operationally, polynomial is used for expressing other functions, due to their usefulness because of such simplicity and versatility. In fact, applications ranges, even to a polynomial basis on n -dimensional space, $(p_1, \dots, p_n) \in K[x, \dots, x_n]$; their growth rate being polynomial also added more to their strength of

approximating fast increasing function, usually expressed and used in setting, for example, polynomial growth class $\mathcal{O}(p(x))$ in computer science. Two applications stand out for such properties: function approximation of arbitrary criteria ℓ , or polynomial interpolation on arbitrary points. Those two are interconnected, though their objective and object of concern is much different.

We can define the application of analytical polynomial on arbitrary function by using the criteria on approximation. Usually, this is expressed in the theory of approximation, and requires a guarantee that polynomial can indeed approximate to arbitrary accuracy, of arbitrary function on specific interval. This is the statement of Weierstrass theorem.

Theorem 6.1 (Weierstrass Approximation Theorem). *Let $f \in C[a, b]$ of all continuously differentiable functions on the closed interval $[a, b]$. Then, for every $\epsilon > 0$, there is a polynomial p such that $\|f - p\| < \epsilon$, for $\|f - p\| = \sum_{i=1}^{\infty} |(f(x_i) - p(x_i))|^a$*

^aWe again note the disparity between $h(x) - c(x)$ or $c(x) - h(x)$ for clarity. In essence, $c(x) - h(c)$ is looking from the perspective of the concept itself. While $h(x) - c(x)$ is comparison on difference from the perspective of the hypothesis.

Here, ϵ is defined to be the *global error* of the polynomial with respect to the arbitrary function, and with uniform approximation. Another variation works on the squared error, such that $\|f - p\| = \sum_{i=1}^{\infty} (f(x_i) - p(x_i))^2$, or the mean absolute

$$\|f - p\| = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n (f(x_i) - p(x_i))$$

Weierstrass Approximation Theorem provides tighter bound for the mean absolute error, but not the square error, or the mean square case. Nevertheless, they are natural measure on the uniform metric space $C(X)$ with measure $d(f, g) = \sup_{x \in X} |f(x) - g(x)|$. If said metric also use the L^p metric instead, which is defined by

$$d_p(f, g) = \left(\int_X |f(x) - f(g)|^p dx \right)^{1/p} \quad (22)$$

we can also see that it is weaker than the bound given by Weierstrass theorem. Generalization of Weierstrass theorem can be made by Stone-Weierstrass theorem instead, of which is state in real space as followed, though generally Weierstrass theorem is sufficed.

Theorem 6.2 (Stone-Weierstrass Theorem (real version)). *Let S be a locally compact Hausdorff space. Let A be the subset of bounded continuous real functions on X , or of subalgebras $\mathcal{C}(X, \mathbb{R})$ which contains a non-zero constant function, then A is uniformly dense in $\mathcal{C}(X, \mathbb{R})$ if and only if it separates points.*

The setting of polynomial interpolation is rather different from the setting of function approximation. The setting of interpolation gives an n samples set $\mathcal{S}$, of which in this case we restrict to $\mathbb{R} \rightarrow \mathbb{R}$. Then, we are required to fit exactly the entire dataset, such that the following setting is sufficed.

Definition 6.3 (Interpolation). *A function class $\mathcal{F}$ is said to **interpolate** a distinct dataset $\mathcal{S}_n = (x_n, y_n)$ of n samples, if there exists $f \in \mathcal{F}$ such that $\mathbb{E} \|S_i - |f(x_i)|\| = 0$. Such f is then called the **interpolating function** of $\mathcal{S}_n$.*

Under such definition, the question is then if the polynomial class $\mathcal{P}_n$ of any given polynomial with the highest degree n is sufficed to approximate such dataset. The answer can be found using unisolvence theorem.

Theorem 6.3 (Unisolvence theorem). *Given $n + 1$ bivariate data points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n) \in \mathcal{S}_n \subset \mathbb{R}^n$, with all x_i distinct, there exists a unique polynomial $p_n(x)$ of degree at most n such that $p(x_i) = y_i$ for $i = 0, 1, \dots, n$.*

In determining such problem, we can also say that there exists a unique n -saddle point function represented here as polynomial, such that to fit n points perfectly by the definition of the saddle point. The process in obtaining such formula is difficult, however, and finding the perfect interpolation polynomial would be computationally significant, and erroneous.

Definition 6.4 (Saddle point, Nie et al. (2021)). Let $X \subseteq \mathbb{R}^n$, $Y \subseteq \mathbb{R}^m$ be two sets (for dimension $n, m > 0$) and let $F(x, y)$ be a continuous function in $(x, y) \in X \times Y$. A pair $(x^*, y^*) \in X \times Y$ is said to be a saddle point of $F(x, y)$ over $X \times Y$ if

$$F(x^*, y) \leq F(x^*, y^*) \leq F(x, y^*), \quad \forall x \in X, \forall y \in Y \quad (23)$$

Polynomial regression is one such analytical solution to function approximation task, using Weierstrass theorem as its basis. From such, we can also observe that polynomial regression, and hence polynomial interpolation, are correlated and can be reduced to each other. Polynomial regression can be reduced to interpolation via stating the criteria such that, denoting it as ℓ , then $\ell(f, p) = 0$, and interpolation can be reduced to regression via the condition that $p(x_i) - y_i \leq \epsilon$ for arbitrary $\epsilon > 0$. Hence, the regression formulae lie between the two extreme: the Weierstrass setting and the interpolation setting.

We then have to identify such extreme and what kind of extreme they represent. Fortunately, we know this as the problem of **transparency**, in the setting of approximation, which certainly correlates to the learning theory. According to such, the Stone-Weierstrass represents the total transparency case, where the function f is known to the approximator with polynomial class. Interpolation represents total black box, where the goal is only to interpolate the sample point by itself. In between such interpretation, there exists a fundamental assumption, hence the structural assumption.

Assumption 6.4 (Transparency). For any bivariate pair (x, y) , we assume each pair depends on and originate from a based concept $y = f(x)$.

Two kinds of knowledge is then available, which correlates to the degree of freedom the observation space can have. First is the knowledge of f . This knowledge of f is encoded directly by the number of sample point and their distribution in certain case. Accordingly, the worst case is as followed.

Conjecture 6.1. If $\forall x$ there exists y corresponding to such, and $(x, y) \in \mathcal{S}$, then we say we have total transparency to the concept f . If there exists only finite x that we can have y corresponding to, then we say we have limited transparency to the concept f . If we assume no knowledge of f , then we say the setting is called the **black-box setting**.

Secondly is the action on the dataset itself. One of the main assumption that is usually observed is that

Assumption 6.5 (Fallacy). We assume every bivariate pair $(x_i, y_i) \in f[\mathcal{S}_n]$, such that $f(x_i) = y_i$.

However, usually, this is not the case, as we have **noise** in addition from the dataset. The origin of the noise cannot be tracked. However, we can interpret what happens when such noise is introduced to the dataset. The noise transitions the transparency of the learned setting, on the field $\mathbb{R}$, by utilizing the dense property of the real space. Assuming we operate on $\mathbb{Z}$. Then, each discrete noise will introduce abnormality point such that no function on $\mathbb{Z}$ will be able to match, up to certain distance on $\mathbb{Z}$. This is because on such space, there exists no point x_1, x_2 , for example, that lies between $d(x_1, x_2) = 1$. If the noise is randomized on ± 1 , then there will exist no function that can approximate such disparity better than a random fluctuating function. However, on $\mathbb{R}$, the real number is inherently dense, such that there will always be certain point x_3 such that $x_3 \in [x_1, x_2]$, by the density of real number. By the distance metric on $\mathbb{R}$, we define $d(X, Y) = |X - Y|$ for distance between two data points. Noise intrinsically increases such distance, such that $d'(X, Y) > d(X, Y)$. Hence, the space in which density can appear increase, thus there then exists more fluctuation, and thus more uncertainty between such two points. This notion can be increased from local uncertainty to global uncertainty, hence the *noisy dataset*.

We then can measure or aggregate some properties of the dataset itself, including the **uncertainty** of the dataset. We define the simple uncertainty of the dataset as followed.

Definition 6.5 (Dataset uncertainty). Given a closed interval $[a, b]$, for the sample set $\mathcal{S}_n = (X, Y) \in [a, b]$, the uncertainty $\mathcal{U}(\mathcal{S}_n)$ is defined as

$$\mathcal{U}(\mathcal{S}_n) = \left[\mathbb{E}_{x_i \in X} (x_{i+1} - x_i)^2 + \mathbb{E}_{y_i \in Y} (y_{i+1} - y_i)^2 \right]^{1/2} \quad (24)$$

We assume X, Y are totally ordered sets.

Hence, the assumption of fallacy assumes $\mathcal{U}(\mathcal{S}_n) = 0$, though in reality it is often $\mathcal{U}(\mathcal{S}_n) > 0$. With this, we can quantify how the dataset is behaving on the scale of the bivariate form typically seen. Noise intrinsically increases this distance, usually by the y -axis. This dataset uncertainty increases with multivariate dataset, for example, $(\mathbf{x}, y)$ of $\mathbb{R}^n \rightarrow \mathbb{R}$. We call the new metric as the *empirical uncertainty metric*. From this, we can have the theorem:

Theorem 6.6 (Uncertainty tradeoff).

Thirdly, is the treatment on such information. Even though information is known, such as from the assumption of transparency and the fallacy conjecture, it relies on the method in which the model uses that make use of such information. Polynomial interpolation effectively removes information about transparency, and thus the interpolation setting is a kind of **black-box absolute setting**. This setting is *extremely brittle*, as to see that if y_i is slightly wrong due to measurement noise, the interpolant oscillates wildly, which is explained by Runge phenomenon (Runge (1901); Corless & Severyi (2018); Boyd (2009)). Probability then arises from such setting naturally, though this particular degree of probabilistic setting must be quantified, which comes from the process that the model simulates upon. Regression is inherently not probabilistic, but treatment about the probabilistic assumption, for example, such that (X, Y) inherently as random variables. We can then separate this into three parts - the data-level probabilistic layer (data uncertainty), the model-level layer, and the algorithm-layer that determines the solution solver within such model. We can define that as followed.

Definition 6.6 (Probabilistic layer). *Given a learning setting with dataset S of bivariate pairs, $h \in \mathcal{H}$, $c \in \mathcal{C}$ as the hypothesis and concept, we have the following conceptual probabilistic setting:^a*

1. *Data-level probability: Sampling assumptions (observation sample points) such that there exists the distribution $\mathcal{D} \sim (X, Y)$, intrinsic data uncertainty $\mathcal{U}(\mathcal{S}_n)$.*
2. *Model-level probability: Structure of the dataset is controlled by the concept structure $\{\theta\} \in c$, c is modelled as a target sampler $c \equiv \mathcal{D}$, knowledge about the structure of c , Bayesian prior-posterior assumptions, $\mathbb{P}(Y | X, \{\theta\})$. For Bayesian polynomial regression, the model-level probability is formulated as*

$$\mathbb{P}(Y | X) = \int \mathbb{P}(Y | X, \theta) \mathbb{P}(\theta) d\theta, \quad \mathbb{P}(\theta_j) = \mathcal{N}(0, \sigma^2) \quad (25)$$

of which assumes aleatoric uncertainty and epistemic uncertainty from the weight.

3. *Algorithm-level probability: Stochastic method, decision conditional (dropout, MCMC), ensembles criteria, Monte-Carlo initialization, variational inference.*

^aCurrently, we have no way to **quantify** the ‘probabilistic degree’ of the system setting itself

Dynamically, in this paper, we would like to investigate this particular junction between uncertainties, to see what happens when the model transparency, the treatment of data, and so on, that exhibits the observations that we would receive from the model itself. The experimental details will infer such result.

Generally, we would be entitled to see particular trajectory specifically designed around the point of interpolating match, or $n = m$ of n degree of freedom, technically, and m data points. After this point, there exist two possibility of dynamic - either the model dilutes, in which case result in *double descent*, or the model fluctuate strongly afterward, leading to *bias-variance emergence* again later on. This conclusion however, would have to be analysed within the current framework of the possible interjection to its fluctuation strength. For example, in the monomial basis $\{x^i\}^n$, any given fluctuation contributes to the entire polynomial. In such case, fluctuation of the other $[0, n - k]$ of arbitrary k and large enough n would be unaccounted for by the scale. Conversely, restrict them to $[1, 0]$ range will reveal something about the system, however will destroy particular properties of the prediction, which is not recommended. Nevertheless, we must conduct the test before giving in any reasonable conclusion.

Previously, we said that we did indeed focus on the interpolation problem. Now, assuming we have given the interpolating algorithm $\mathcal{A}$ to approximate the interpolation, what will this look like? Firstly, solving this is *very computationally expensive*, because it is a dense, $(n + 1) \times (n + 1)$ linear system, and even more so if we consider the overparameterized outcome. However, this can be reduced further down if we attempt to solve them using convergence bound, but will be still very expensive. That and incremental soft interpolation is somewhat out of the question since it will be kind of the same. Aside from the insight, we would still have to know what to do. Then there exists two ways - we have some version of this to optimize of simply evaluating

by gradient descent optimization, or, we can use somewhat analytical fit for the case of soft interpolation modification.

A theory on such problem can be obtained of its mechanics, as will be a fairly intuitive way in consideration of the theory of interpolation, as well as the technique of interpolation by itself. For a fixed-range interpolation threshold problem, a hard-encoded interpolation solution requires n -degree polynomial, of which has $n + 1$ degree of fitness, to fit $n + 1$ distinct points. Suppose all points are distinct of the set S which is finite, then this is the case. If we switch to the problem of interpolation but induce a more forgiving bound, there exists a possibility that it induces a relative *density averaging path* for the points, especially if we are considering the case where $p > n$ for p saddle points or degree of fitness, and n the number of point to interpolate. In a more nuanced formulation, we have

$$\text{Flatten} \left(\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^p \\ 1 & x_1 & x_1^2 & \cdots & x_1^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_p & x_p^2 & \cdots & x_p^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^p \end{bmatrix}_{n \times p} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_p \end{bmatrix} - \begin{bmatrix} f(x_0) \\ f(x_1) \\ f(x_2) \\ \vdots \\ f(x_n) \end{bmatrix} \right) \leq \epsilon \quad (26)$$

or equivalently, $\text{Flatten}(\mathbf{X}_{n \times p} \mathbf{w}_n - \mathbf{Y}_n) \leq \epsilon$ where $\text{Flatten}(\cdot)$ is the compression of vector to scalar, or so $\text{Flatten} : \mathbb{R}^n \rightarrow \mathbb{R}$. This essentially is the problem of solving n points approximation, toward using only p saddle points available. If $\epsilon = 0$, it is well known that interpolation is impossible. However, if $\epsilon \neq 0$, there are possibilities, depends on the shape of the dataset, that there will exist a data-dependent property or more that indict a particular *density preference* toward those saddle points - or rather, the act of density averaging. This is reflected rather implicitly in the structure itself. In machine learning, the error measure is usually the mean average, such is to say that the criterion now would look more like $\mathbb{E}[\text{Flatten}(\mathbf{X}_{n \times p} \mathbf{w}_n - \mathbf{Y}_n)] \leq \epsilon$. Formally, we have the following hypothesis

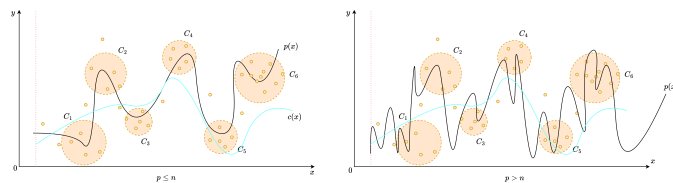
Hypothesis 6.1 (Polynomial double descent). *For a polynomial $\{a_i x^i\}_n$ on Vandermonde basis, approximating on the set $\{(x_k, y_k)\}_m$, then*

1. *For $n < m$, we expect the polynomial form factor to be inherently sparse, such that for an iterative approximation, for each p_i saddle point of the polynomial expansion form, there exists an ϵ -ball B_ϵ for $\epsilon > 0$ with at least one data point satisfying $(x_k, y_k) \in B_\epsilon$.*
2. *For $n = m$, we expect the polynomial to be able to fit all points, up to hard interpolation threshold such that $\hat{R}_S \approx 0$.*
3. *For $n > m$, we expect the polynomial form factor to be locally dense then turning to globally dense, over random ensembles. That is, for every point (x_k, y_k) there exists a β -ball B_β such that there is at least one saddle point of the polynomial belong to the ball.*

The shift that will be hypothesized to appear here is expected to be from sparse in the polynomial term sense, out to globally dense with enough model complexity, over an iterative process. One also main point of clarity is, at certain point right now, we haven't discovered the effect of error deviation, yet. For such, we theorize the following.

Conjecture 6.2 (Double descent). *Double descent happens, for a polynomial class model $h_n \in \mathcal{H}_p$, is upper bounded to certain error deviation $\epsilon > 0$ from the mean only of the generalization (test) dataset verification.*

Figure 6: Intuition of Hypothesis 6.1. Effectively, in the overparameterized range ($p > n$), the problem turns into soft interpolation, where the polynomial is possible to be dense enough such that relative error remains internally low. This can also happen if the dataset itself is intrinsically large.



That is, the disparity between the test and training dataset helps in such regard. If the two dataset is partitioned in the sense that their deviation is practically the same (which can be done using synthetic generation for a

control group), then this will amount to not much of it. After all, it will be the consequence of the algorithm and error landscape interpretation used.

For this, we propose another speculative observation beside such, as to consider adjourned transform of the Vandermonde basis to the Newtonian or similar basis, in which the effects and motion can be easily observed, as least easy. This is because aside from mapping down to $[-1, 1]$, normal polynomial on monomial basis gives suboptimal result because of intrinsic high-degree term domination.

This will become more relevant later on in the analysis.

6.0.1 POLYNOMIAL MODEL STRUCTURE

Now, for mathematical analysis and experimental design, we would like to state the current main structure of a polynomial model. Specifically, in such example, we are concerned of the shallow polynomial neural network, of which is stated as such in Arjevani et al. (2025). Notice that it is defined on the normal Vandermonde basis, where an expression of such polynomial model under Newtonian or Lagrangian basis is not so clear-cut of an analysis.

Definition 6.7 (Shallow polynomial neural network). *A shallow polynomial is a neural network of one layer that are functions $f_{\mathcal{W}}$, defined such as: $\mathbb{R}^d \rightarrow \mathbb{R}$ of the form*

$$f_{\mathcal{W}}(x) = \sum_{i=1}^r \alpha_i (w_i \cdot x)^d, \quad \mathcal{W} = (\alpha, w_1, \dots, w_k) \in \mathbb{R}^r \times (\mathbb{R}^n)^k, \quad (27)$$

where $d \in \mathbb{N}$.

A modification is then available, in which we simply add $b(\cdot)$ for random initialization process. We note that this introduces certain flexibility to the model initial interpolation curvature.

Definition 6.8 (Polynomial neural network, general). *The polynomial network based on Model 6.7 is modified as*

$$f_{\Theta}(x) = \sum_{i=1}^d \theta_i (\mathbf{w}^{\top} x)^i + b, \quad \Theta = (\{\theta_i\}, \{w_k\}, b) \quad (28)$$

From this, we can state the model within a testing scenario listed as such, to fully capture the structural foundation of the related testing model class. For the notion of complexity, naively, we employ two kinds of complexity measure. First is the usual complexity measure via parameter counts. Second, is the expressive complexity on the basis of the linear transformation space. That is, an expressive complexity that take into account the fact that $\mathcal{O}(wx^n)$ will always be stronger than $\mathcal{O}(wx^{n-1})$ for $x > 1$. All of which can be contained in a singular expression, as seen in definition 6.9. However, we would have to analyse what is happening inside the metric itself. In the equation for expressive complexity, the first term is the normalized linear parameters θ_i fixing and controlling the shift of the polynomial inlining parameter, the second term is the sum of weight-aware complexity with degree factor, and the degree factor over operating range $\|x_{max} - x_{min}\|$ of the dataset, for the degree complexity. The final term is the last weight-aware complexity for the dominating degree n of the polynomial function. Let us begin with the gauge on coefficient norm, i.e. the term $\|\alpha_d\| + 1$. Here, we set 1 as baseline, and the arbitrary norm on α there coefficient effect. The terms coefficient in this expression matters because, for a function $f : [a, b] \rightarrow \mathbb{R}$, the Lipschitz constant L for polynomial f goes

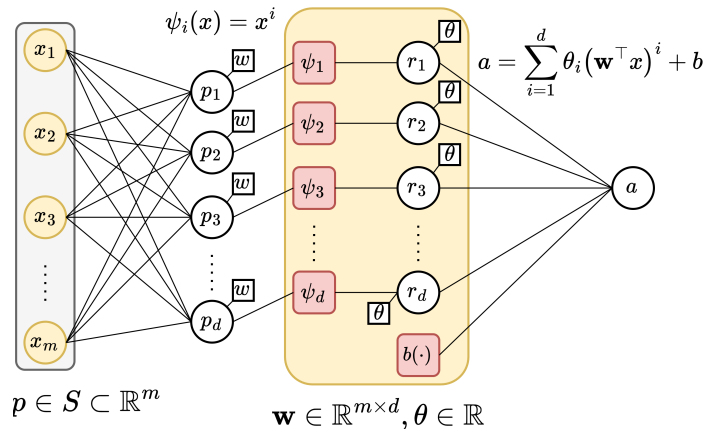


Figure 7: Illustration of polynomial model in definition 6.8.

by

$$\begin{aligned}
L_p &= \sup_{x \neq y} \frac{|f(x) - f(y)|}{|x - y|} \\
&= \sup_{x \neq y} \frac{|f_\Theta(x) - f_\Theta(y)|}{|x - y|}, \quad f_\Theta(x) = \sum_{i=1}^d \theta_i (\mathbf{w}^\top x)^i + b \\
&\leq \sup_{x \in [a, b]} \|f'_\Theta(x)\|_2 \\
&= \sup_{x \in [a, b]} \left\| \sum_{i=1}^d i \theta_i (\mathbf{w}^\top x)^{i-1} \mathbf{w} \right\|_2 \\
&\leq \|\mathbf{w}\|_2 \sup_{x \in [a, b]} \sum_{i=1}^d i |\theta_i| |\mathbf{w}^\top x|^{i-1} \\
&\leq \|\mathbf{w}\|_2 \sum_{i=1}^d i |\theta_i| \max\{|\mathbf{w}^\top a|, |\mathbf{w}^\top b|\}^{i-1}.
\end{aligned} \tag{29}$$

Where $\|\mathbf{w}\|_2$ is the Euclidean ℓ^2 norm. The magnitude of $f'_\Theta(x)$ depends directly on the coefficient size α_i , and also on how big is x on the domain. All of this mean for large expression and value range on f_Θ , the Lipschitz constant of that model is accordingly large. Defining α -norm that way is a cheap proxy for Lipschitz constant, since it gauges stability, and rate of change of the functional expression itself.

Definition 6.9 (Polynomial expressive complexity). *For $x > 1$ or $\mathbf{x}_i > 1$ of $\mathbf{x} \in \mathbb{R}^n$ as input. Denote the polynomial class as $\mathcal{P}_n$ of their highest degree n , then we define the total expressive complexity of $p \in \mathcal{P}_n$ as followed.*

$$\mathbb{C}_E(p) = \mathbb{E} \left[\|\alpha\|_d + 1 + \frac{\sum_{i \in S \setminus \{p_n\}} w_i i}{\|p\| - p_n} + \frac{1}{\|\bar{x} - \underline{x}\|} \left(\frac{\sum_{i \in S} i}{\|p\|} + w_n n \right) \right], \tag{30}$$

where $S = \{p_j\}$, $\|p\|$ denotes the amount of different degree separable terms and $\|x_{max} - x_{min}\|$ is the observable range of the dataset.

6.0.2 TESTING RESULT

The testing scenario will be as followed. The following set of parameters are required for controlling the testing case, aside from different system measures. For parameter error, we reuse equation ?? but with degree aware terms.

We note a few problems with both interpolation interpretation and polynomial regression of higher-degree terms. Firstly, is the intrinsic interpolation error and numerical, software issue error. Higher degree polynomial can be represented as solving for a wide Vandermonde system of n degrees for m sample points. Using monomial basis for interpolation, either for soft interpolation also exhibits instability and numerical garbage within the operations, as indicated by Shen & Serkh (2025). They can also be a potential source of double descent, as the majority of models tested around this range are not coded effectively for handling large data operations as such, and artifacts from such choices might be taken as the indication of double descent in some particular cases.

Aside from it, and aside from the fully-connected case, we have the particular testing scenario that can be expected of the ease of implementation. First, is the mundane implementation that is of gradually increasing degree mismatch. That is, for polynomial of h_p and c_p , one requires only d_h and d_c to figure out the structure as there will exist exactly $d + 1$ with the constant term, terms of which of finite many degrees. The second case is where for p chosen randomly, such that aside from such p entry, then the other degree or nodes of input are simply scattered dependent nodes, linear algebraically. That is, for a uniformly constructed d -degree network, choose at random p particular degree to be fixed, while the other of $d - p$ are assigned randomly of belonging to some $p_i \in \{p\}$ nodes given the random choice on p above. Linearly algebraically, scattered and dependent nodes then mean for example, node 4 might use node 35's degree, and vice versa (if they were both non-fixed). The dependency structure is deliberately randomized rather than organized. For overparameterized case, we simply design the hypothesis by the following line of justification. For a mapping $\mathbb{R}^d \rightarrow \mathbb{R}$, the mapping $\mathbb{R}^{d+n} \rightarrow \mathbb{R}$ for $n = 1$ requires additional information, since the source provision does not contain information to handle additional n dimensional input. Hence, we design it the following. For the hypothesis

Table 1: Controllable parameters and hyperparameters for polynomial class analysis

Information	Notation	Details
Hypothesis model	$h[\mathcal{W}_h, \mathbf{H}_h] \in \mathcal{H}$	Hypothesis model. For simplicity, it contains $\mathcal{W}$ as the configuration parameter set, and $\mathbf{H}$ the fixed hyperparameters
Concept model	$c[\mathcal{W}_c, \mathbf{H}_c] \in \mathcal{C}$	Concept or target model. Similar configuration as the hypothesis model
Complexity measure(s)	$\{\mathcal{C}(\cdot)\}$	Set of complexity measure of choice for any arbitrary model taken in.
Sample count	m	The number of sample available for hypothesis model of c , for $m = \mathcal{S} $ dataset
Hypothesis dimension (hyperparameter)	$[d_h]$	An explicit hyperparameter contains the list of all in-testing polynomial degree for the hypothesis model.
Concept dimension (hyperparameter)	d_c	An explicit hyperparameter of the concept's true dimension. Theorized such that $d_c = d_{h,i}$ for arbitrary i will reduce to possible low-error problem.
Data noise	$\epsilon_{\mathcal{S}}$	Dataset-wide noise with relative strength to the value of the sample point. This is to reduce vanishing noise with higher-degree polynomial sample points.
Learning parameters	epoch, lr , bs	Two main parameters for gradient-descent, iterative-based algorithm - the epoch, the learning rate, and the batch size.

Notes: We output to $\{\mathbb{E}[\ell]\}$ as the expected value of the error measure on h, c . There are a few things possible, but under approximation setting, we use MSE-on-parameter and MSE-on-prediction for two different insights: internal representation and external representation.

h , such unit of which does not fit to the original input space, the *internal input space* that h ‘hypothesize’ contains, for the other n entries, $q \leq d$ per unit input. That is, each input of n overparameterized case receives at random, q inputs can be received, for example, $w_{n1}x_j + w_{n2}x_k$ for k, j specific input numbering. By the simplicity argument, then $q = 1$ for the simplest case. However, this creates a skewed polynomial network, of which informally, the provided structural bias or additional information, is the fact that there exists overparameterization. To prevent this, instead, the final case considers the most general structure — for each unit, each one can take q inputs controlled as hyperparameter of the network, then the set $\{p\}$ of chosen degree influence their degree choice, and is uniform if the sampling process it is not included.

7 AXIOMATIC FORMALIZATION

Concurrent theories are inherently unstable and with many variations, making them ill-suited for instant application on analysis of different models without losing major information about the structural details and expressions. Hence, we propose a novel analysis on the basis of the axiomatization of the learning setting, under various structural assumptions.

7.1 LEARNING THEORY FORMALISM

The classical theory of learning simply does not have much to offer in terms of a unified perspective, in which important factors are analysed of its realization. Specifically speaking of such, classical theory does do worse in many parts, for example, the aspect of transparency given the model, supervisor and the concept, of which their roles are not fully understood or formulated correctly, both in controlled setting and extension to replication of real life examples. Thereby, the problem requires a newer theory of learning to come forward. A new learning theory must extend beyond what constitute the basic learning model, such as the idealized PAC-learning, and statistical learning setting in general. More specifically, we want to design a learning theory that is more akin to **modelling theory** of its internalization, but also attain the learning of machine learning theory.

Within the constraints of such theoretical framing, we can then construct a different notion in which the generalization and the empirical learning problem can possibly relate to each other, at least from the statistical framework. Let us consider the problem of mathematical modelling in such sense. Beside the model class construction, which will be discussed in further sections,

7.2 THE IDEA OF NEURAL NETWORK FORMALISM

Previously, we have been discovering, examining past results and testing on models of different types, generally classical models. While decisive, each modelling setting requires different analysis and is often not so streamline. Hence, we propose a general analysis on the structure of a feed-forward style neural network architecture, as usual, for further analysis and identification.

A neural network (Goodfellow et al. (2016); Zhang (2023)) N is typically specified by the dual $(L, [S])$ where L is the number of processing layer of a neuron, and $[S]$ is the list of neurons configuration per layer - for example, the number of neurons per layer, and so on. For such network, the definition can be simple as followed

Definition 7.1 (Standard multilayer network, Zhang et al. (2023)). *We define a K -layer fully-connected deep neural network with real-valued output. Let $m^{(0)} = d$ and $m^{(K)} = 1$ for m the width of the network, and d the shape of the input vector. We then recursively define:*

$$x_j^{(0)} = x_j \quad (j = 1, \dots, m^{(0)}), \quad (31)$$

$$x_j^{(k)} = h \left(\sum_{j'=1}^{m^{(k-1)}} \theta_{j,j'}^{(k)} x_{j'}^{(k-1)} + b_j^{(k)} \right) \quad (j = 1, \dots, m^{(k)}), \quad k = 1, 2, \dots, K-1 \quad (32)$$

$$f(x) = x_1^{(K)} = \sum_{j=1}^{m^{(K-1)}} u_j x_j^{(K-1)} \quad (33)$$

where the model parameters can be represented by $w = \{[u_j, \theta_{j,j'}^{(k)}, b_j^{(k)}] : j, j', k\}$ with $m^{(k)}$ being the number of hidden units at layer k ; $\theta \in \mathbb{R}^m$ the weight of the neuron.

Here, each neuron is denoted by their expression as $f(\mathbf{W}^\top \mathbf{x} + b)$ for the activation function f . We can simplify a lot of those structures above into the neural network schemes. For example, the linear function class case can be simplified to a single $(2, [n, 1])$ neural network, such as:

$$x_j^{(0)} = x_j \quad (j = 1, \dots, n) \quad (34)$$

$$x^{(1)} = \sum_{j'=1}^n w_{j'} x_{j'}, \quad (35)$$

without the activation function h . The polynomial neural network can be represented by

$$x_j^{(0)} = x_j \quad (j = 1, \dots, m^{(0)}), \quad (36)$$

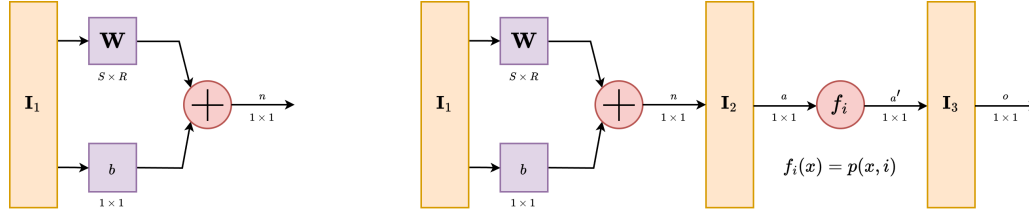


Figure 8: Flow diagram of the shallow neural network structure in linear and polynomial case, following the notation of Demuth et al. (2014) in *Neural Network Design*. The Support Vector Machine flow diagram looks similar to the case of polynomial, except a decision function at the end of the output, and changing the function f into the kernel function $K(x', x)$.

$$x_j^{(1)} = h \left(\sum_{j'=1}^{m^{(0)}} \theta_{j,j'}^{(1)} x_{j'}^{(0)} \right) + b_j^{(k)} \quad (j = 1, \dots, m^{(1)}), \quad h(x_j) = x_j^j \quad (37)$$

$$f(x) = x_1^{(2)} = \sum_{j=1}^{m^{(1)}} u_j x_j^{(1)} \quad (38)$$

for a neural network of shape $(3, [n, n, 1])$, and of polynomial custom scaling function. For support vector machine, this is even harder to do, since its definition varies between different notion of the model itself. However, it is interesting because we can treat it as an arbitrary generation of a neural network structure. There are papers about formalizing SVM in the sense of neural network, including making it to be an infinitely-wide shallow neural network, for example, Chen et al. (2021); Wang et al. (2019); Zhang & Wang (2014), however, the structure that is formed from the SVM learner can be expressed as followed:

Definition 7.2 (Support vector machine setting). *For any given optimization problem on input space $x_i \in \mathbb{R}^d$, and label $y_i = \{\pm 1\}$, the shallow neural network resultant from the training procedure is always of the form:*

$$x_j^{(0)} = x_j \quad (j = 1, \dots, m^{(0)}), \quad (39)$$

$$x_j^{(1)} = \sum_{j' \in m^{(1)}} \theta_{j,j'}^{(1)} x_{j'}^{(0)} K(x_{j'}^{(0)}, x) \quad (40)$$

$$f(x) = \text{sign}(x_1^{(2)}) \quad (41)$$

for a specific kernel $K(x, x')$.

Those three fundamental conversions to neural network raise interesting insight into the neural network formalism. For linear and polynomial shallow neural network, we design the network and then provide it with a training scenario, evaluated of the proxy generalization. SVM however can be considered an adaptive data construction, where the neural network is constructed from the dataset or observation set itself, using minimization on RKHS and with constraints. This prompts exactly as such, on one side the pre-optimization construction, and one is the post-construction optimization of a neural network architecture. This interpretation is surprisingly useful when talking about neural network's unit-like structure. Hence, from such evidence prompts us to analyse a neural network, not in only the sense of managing its layers, but also by its unit components together.

This way of analysing unit-wise gives us much more interpretability, and also potential for a newer kind of network specification. As far as we are concerned of, a neural network can represent an active *mathematical modelling system*. This then indicates, for example in adversarial learning case between the target neural network tailored to specific concept, and the learning hypothesis network, then the process of learning itself can be loosely interpreted as a concept of mathematical modelling representation learning, even in the case of black-box model, and not just simply phenomenological modelling of statistical data. To do this, we will examine the structure from the perspective of a **generator**.

7.3 SUPPORT VECTOR MACHINE

Support vector machine, first formally originated from Vapnik (1999), is a model inherently, specifically defined for pattern recognition task, and binary classification via an *optimal separating hyperplane*. There are two variants for SVM, namely, for linear and nonlinear hyperplane, but the main general aspect of SVM is its reversal of the direction on model building - instead, it aims to minimize the structure of a model, using the data-dependent structure, or *support vectors*.

The SV machine implements the following two precursor ideas: It maps the input vectors x , supposed of the setting, into a high-dimensional feature space Z through some nonlinear mapping, chosen a priori. In this space, an optimal separating hyperplane is constructed. By statistical learning theory, Vapnik restricted the function class (as for infinite hypothesis it is impossible to learn) to the class of hyperplanes by

$$\langle \mathbf{w} \cdot \mathbf{x} \rangle + b = 0; \quad \mathbf{w} \in \mathbb{R}^n, b \in \mathbb{R} \quad (42)$$

which $\mathbf{w}$ is the controlling weight, $\mathbf{x}$ is the input space to the space of all binary $\{-1, +1\}$ category. Hence, this basically divide the input space into two: one part containing vectors of the class -1 and the others being $+1$. If there exists such a thing, then it is said to be *linearly separable*. To find the class of a particular vector $\mathbf{x}$, we use the following decision function

$$f(\mathbf{x}) = \text{sgn}[\langle \mathbf{w} \cdot \mathbf{x} \rangle + b] \quad (43)$$

This is called the **hyperplane classifier** class³. The above form of finding final classification can then be presented in dual form, which then depends only on dot products between vectors, as

$$f(\mathbf{x}) = \text{sgn} \left(\sum_{i=1}^{\ell} y_i \alpha_i \langle \mathbf{x} \cdot \mathbf{x}_i \rangle + b \right) \quad (44)$$

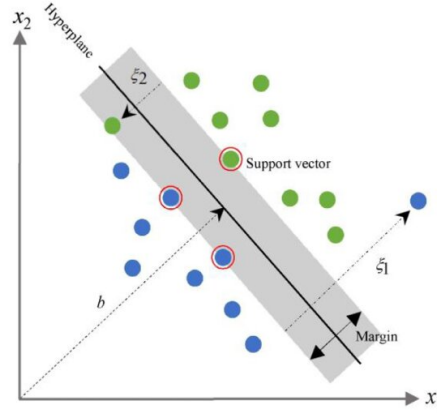


Figure 9: Illustration of support vector machine (SVM) on the observable latent space, with the goal to generalize the optimal separation hyperspace in linear separable data or via a proxy kernel space, as introduced. Reused from Khouma et al. (2019).

where $\alpha_i \in \mathbb{R}$ is a real-valued variable that can be viewed as a *measure* of how much informational value $\mathbf{x}_i$ has (for x_i the support vectors we get from training)⁴. The second idea is of the *kernel method*. This particular method, useful and well-known of the time SVM was created, gain a more prominent position among other techniques, including neural network. Specifically, this method is characterized by the mapping of the input vectors into a richer (usually high-dimensional) feature space where they satisfy *linear separable* criteria. This prompted the possibility to solve nonlinear problem through such mapping, and indeed yields a nonlinear decision surface in the input space, which is linear in the feature space. A **kernel** (function) is then a function

³As can be understood simply from such, there exists many hyperplanes that can correctly classifies the classes. It has been then shown that the hyperplane that guarantees the best generalization problem performances is the one with the maximal margin of separation between two classes (Cristianini & Shawe-Taylor (2000)).

⁴The *support vector* of a particular hyperplane $\mathbf{y}$ is the collections of all points that lies closest to the hyperplane, which is specified as condition for maximization on both side of maximal margin vector machine.

$k(\mathbf{x}, \mathbf{y})$ that given two vectors in input space, return the dot product of their images in feature space such that $k(\mathbf{x}, \mathbf{y}) = \langle \phi(x), \phi(y) \rangle$ for the nonlinear mapping ϕ . The general form of SVM is then

$$f(\mathbf{x}) = \text{sgn} \left(\sum_{i=1}^{\ell} y_i \alpha_i k(\mathbf{x} \cdot \mathbf{x}_i) \right) \quad \ell \leq n \quad (45)$$

for any particular final categorization. One of the major problem for analysing SVM, is in the properties of it potentially having infinitely many parameters, depends on the size of the dataset. This makes the curve of both bias-variance and double descent not applicable in such regard - infinitely many parameters' complexity, yet still finite complexity class⁵.

7.4 FORWARD-BACKWARD CONSTRUCTION

With respect to the observable space, SVM and neural network, we can call the construction scheme of SVM a type of *backward construction* from data $\mathcal{S}$. Or rather, the following algorithm represents SVM construction problem:

Factory 3: Support Vector Machine - backward construction.

Data: $(x_i, y_i) \in \mathcal{S}, y_i = \{\pm 1\}$, hinge objective $\ell(y, f(x))$, $\nabla(\cdot)$ differentiable gradient field on ℓ .

Result: Expected optimized model to the minimal error on the objective $\ell(\cdot, \cdot)$, dependent on $\mathcal{S}$.

begin

$d \leftarrow \dim |\mathcal{S}|;$

$n \leftarrow |\mathcal{S}|;$

$\ell(y, f(x)) = \max(0, 1 - yf(x));$

Kernalization $\phi : \mathbf{x} \rightarrow \mathbf{x}'$;

$f(\mathbf{x}) \leftarrow \mathbf{w}^\top \phi(\mathbf{x}) + b$;

Objective

Attain the minimization

$$\min_{w, b \in \mathcal{F}} \frac{1}{2} ||w||^2 + C \sum_{1 \leq i \leq n} \ell(y_i, f(x_i)) \quad (46)$$

Ideally for $y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b) \geq 1, \forall i, y_i \in \mathcal{S}$, penalized by ℓ ;

end

return Optimized model SVM(f) on objective ℓ of ∇ field.

Such a factory is reversed inherently, and with its formulation constrained in the form $\mathbf{w}^\top \phi(\mathbf{x}) + b$, of which is equivalent to a non-linear shallow neural network with infinite width (Wang et al. (2017); Chen et al. (2021)), but is not a full-fledged neural network in technicality, hence it is only a functional equivalence. That is, such result only indicates that for $N \rightarrow \infty$ of the number of neurons of a single-hidden layer neural network, $\sum_{j=1}^N a_j(w_j^\top x + b_j)$, then

$$\text{NN}(x) \approx \text{SVM}(x) = \sum_i^{\ell} \alpha_i k(x, x_i), \ell \leq n \quad (47)$$

The support vector, ℓ list, is inherently implicit of the factory itself. On the other hand, typical deep learning or neural network constructions are rather *forward-constructed*, that is, the model is in a sense, prefabricated then tested of its efficacy, and thus the effective performance of the model is limited by iterations of its restricted frame. We can easily visualize the factory of which neural network are constructed, different of the SVM factory case:

⁵From such observation, it would seem imperative that the notion of complexity based solely on the parameter counts of certain model construction is ill-equipped for certain flavours of machine learning model, in this case support vector machine (in the same type is the Gaussian Mixture Model). No double descent has been observed by empirical reports record per the author's knowledge.

7.5 THE COMPONENT PERSPECTIVE

Let us look at neural network in a more component perspective, or rather, similar to a **resource** combination and advancement game. Let us play this game within the following sense. Generalizing neural network means working with component and unit-type framework, since much of our current understanding relies on the basis of hard-coding the flow of operations (feed-forward), external update on global space (backpropagation), fixed domain of construction into horizontal scaffolding and vertical stretching (layers and neurons per such unit). However, to even now of this point, we haven't turned our attention to it of the fullest. In fact, the individual neuron itself is made up of components. If one want a systematic structural analysis, one will have to consider the components' description and metric as it is, before going forward in the interpretation thereof. Furthermore, perhaps there would be a way to link the learning process, right now being interpreted as external of the model, into the system that the model is considered. Such requires a prototype of a new kind of consideration in modelling theory, and as well to the fact that it would be well-supported by many bounds that are usually there in statistical learning theory, which makes this implication a non-trivial yet natural argument.

REFERENCES

- Panos Achlioptas. Stochastic Gradient Descent in Theory and Practice. *Lecture note, Stanford's AI*.
- Ben Adlam and Jeffrey Pennington. Understanding double descent requires a fine-grained bias-variance decomposition, 2020. URL <https://arxiv.org/abs/2011.03321>.
- Madhu S. Advani and Andrew M. Saxe. High-dimensional dynamics of generalization error in neural networks, 2017. URL <https://arxiv.org/abs/1710.03667>.
- Oskar Allerbo. Changing the kernel during training leads to double descent in kernel regression, 2025. URL <https://arxiv.org/abs/2311.01762>.
- Yossi Arjevani, Joan Bruna, Joe Kileel, Elzbieta Polak, and Matthew Trager. Geometry and optimization of shallow polynomial networks, 2025. URL <https://arxiv.org/abs/2501.06074>.
- Mikhail Belkin, Siyuan Ma, and Soumik Mandal. To understand deep learning we need to understand kernel learning, 2018. URL <https://arxiv.org/abs/1802.01396>.
- Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine learning practice and the bias-variance trade-off. *Proc. Natl. Acad. Sci. U.S.A.*, 116(32):15849–15854, August 2019. ISSN 0027-8424, 1091-6490. doi: 10.1073/pnas.1903070116. URL <http://arxiv.org/abs/1812.11118>. arXiv:1812.11118 [cs, stat].
- Mikhail Belkin, Daniel Hsu, and Ji Xu. Two models of double descent for weak features. *SIAM Journal on Mathematics of Data Science*, 2(4):1167–1180, January 2020. ISSN 2577-0187. doi: 10.1137/20m1336072. URL <http://dx.doi.org/10.1137/20m1336072>.
- JP. Boyd. Divergence (runge phenomenon) for least-squares polynomial approximation on an equispaced grid and mock-chebyshev subset interpolation. *Applied Mathematics and Computation*, 213(1):292–303, 2009. doi: 10.1016/j.amc.2008.12.087. Study of divergence behavior and mitigation by Mock-Chebyshev grid.
- David Brellmann, Eloïse Berthier, David Filliat, and Goran Frehse. On double descent in reinforcement learning with LSTD and random features. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=9RIbNmX984>.
- Gavin Brown and Riccardo Ali. Bias/variance is not the same as approximation/estimation. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=4TnFbv16hK>.
- Yilan Chen, Wei Huang, Lam M. Nguyen, and Tsui-Wei Weng. On the equivalence between neural network and support vector machine. *arXiv preprint arXiv:2111.06063*, 2021. URL <https://arxiv.org/abs/2111.06063>.
- Vladimir Cherkassky and Eng Hock Lee. To understand double descent, we need to understand vc theory. *Neural Networks*, 169:242–256, 2024. doi: 10.1016/j.neunet.2023.10.014. URL <https://doi.org/10.1016/j.neunet.2023.10.014>.
- Robert M. Corless and Leili Rafiee Sevyeri. The runge example for interpolation and wilkinson’s examples for rootfinding. *arXiv preprint*, 0(arXiv:1804.08561), 2018. URL <https://arxiv.org/abs/1804.08561>. Modern analysis of Runge’s phenomenon and conditioning.
- Nello Cristianini and John Shawe-Taylor. An introduction to support vector machines and other kernel-based learning methods. 2000. URL <https://api.semanticscholar.org/CorpusID:60486887>.
- Stéphane d’Ascoli, Levent Sagun, and Giulio Biroli. Triple descent and the two kinds of overfitting: where & why do they appear? In *Advances in Neural Information Processing Systems*, volume 33, pp. 3058–3069. Curran Associates, Inc., 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/1fd09c5f59a8ff35d499c0ee25a1d47e-Abstract.html>.
- Xander Davies, Lauro Langosco, and David Krueger. Unifying Grokking and Double Descent, March 2023. URL <http://arxiv.org/abs/2303.06173>. arXiv:2303.06173 [cs].
- Howard B. Demuth, Mark H. Beale, Orlando De Jess, and Martin T. Hagan. *Neural Network Design*. Martin Hagan, Stillwater, OK, USA, 2nd edition, 2014. ISBN 0971732116.
- Pedro Domingos. A unified bias-variance decomposition and its applications. In *Proceedings of the 17th International Conference on Machine Learning (ICML-2000)*, pp. 231–238. Morgan Kaufmann, 2000a. URL <https://homes.cs.washington.edu/~pedrod/papers/mlc00a.pdf>.
- Pedro Domingos. A unified bias-variance decomposition for zero-one and squared loss. In *Proceedings of the 17th National Conference on Artificial Intelligence (AAAI-2000)*, pp. 564–569. AAAI Press, 2000b. URL <https://homes.cs.washington.edu/~pedrod/papers/aaai00.pdf>.

- Pedro M. Domingos. A Unifeid Bias-Variance Decomposition and its Applications. In *Semantic Scholar*, June 2000c. URL <https://www.semanticscholar.org/paper/A-Unifeid-Bias-Variance-Decomposition-and-its-Domingos/eled9d24db5e8f7ab326aeb797e965a94f5ad6d3>.
- George Casella E. L. Lehmann. *Theory of Point Estimation*. Springer Texts in Statistics. Springer-Verlag, New York, 1998. ISBN 978-0-387-98502-2. doi: 10.1007/b98854. URL <http://link.springer.com/10.1007/b98854>.
- Stuart Geman, Elie Bienenstock, and René Doursat. Neural networks and the bias/variance dilemma. *Neural Computation*, 4(1):1–58, 1992. doi: 10.1162/neco.1992.4.1.1.
- Ian Goodfellow, Yoshua Bengio, Aaron Courville, and Yoshua Bengio. *Deep learning*, volume 1. MIT Press, 2016.
- Bruce Hajek and Maxim Raginsky. *Statistical Learning Theory*, volume 1. 2021. URL <https://maxim.ece.illinois.edu/teaching/SLT/>.
- Ouns El Harzli, Bernardo Cuenca Grau, Guillermo Valle-Pérez, and Ard A. Louis. Double-descent curves in neural networks: a new perspective using gaussian processes, 2023. URL <https://arxiv.org/abs/2102.07238>.
- Rebekka Heckel and Murat Yilmaz. Early stopping in deep networks: Double descent and how to eliminate it. arXiv preprint, 2020. URL <https://arxiv.org/abs/2007.10099>. discusses epoch-wise double descent and mitigation strategies.
- Erdong Hu. An overview of double descent and overparameterization. *independent*, (0), 2021.
- Arthur Jacot, François Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. Kernel view of wide-network behavior.
- Gareth James, Trevor Hastie, Robert Tibshirani, and Daniela Witten. *An introduction to statistical learning : with applications in R*. New York : Springer, [2013] ©2013, 2013. URL <https://search.library.wisc.edu/catalog/9910207152902121>.
- Ousmane Khouma, Idy Diop, Papa Alioune Fall, Mamadou Lamine Ndiaye, Sidi M. Farssi, Abdou M. Oussamatou, and Birahime Diouf. Novel classification method of spikes morphology in eeg signal using machine learning. *Procedia Computer Science*, 148:70–79, 2019. doi: 10.1016/j.procs.2019.01.010.
- Marc Lafon and Alexandre Thomas. Understanding the Double Descent Phenomenon in Deep Learning, March 2024. URL <http://arxiv.org/abs/2403.10459>. arXiv:2403.10459 [cs, stat].
- Xinyue Li and Rishi Sonthalia. Least squares regression can exhibit under-parameterized double descent. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 25510–25560. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/2d43f7a61b57f83619f82c971e4bddc0-Paper-Conference.pdf.
- Chris Yuhao Liu and Jeffrey Flanigan. Understanding the role of optimization in double descent, 2023. URL <https://arxiv.org/abs/2312.03951>.
- Fanghui Liu, Zhenyu Liao, and Johan A. K. Suykens. Kernel regression in high dimensions: Refined analysis beyond double descent, 2021. URL <https://arxiv.org/abs/2010.02681>.
- Yifan Luo and Bin Dong. Double descent of discrepancy: A task-, data-, and model-agnostic phenomenon, 2023. URL <https://arxiv.org/abs/2305.15907>.
- Song Mei and Andrea Montanari. The generalization error of random features regression: Precise asymptotics and double descent curve. *arXiv preprint arXiv:1908.05355*, 2019a.
- Song Mei and Andrea Montanari. The generalization error of random features regression: Precise asymptotics and double descent curve. *arXiv preprint*, 2019b. URL <https://arxiv.org/abs/1908.05355>.
- Song Mei and Andrea Montanari. The generalization error of random features regression: Precise asymptotics and double descent curve, 2020. URL <https://arxiv.org/abs/1908.05355>.
- Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of Machine Learning*. The MIT Press, 2012. ISBN 026201825X.
- Preetum Nakkiran. More data can hurt for linear regression: Sample-wise double descent, 2019a. URL <https://arxiv.org/abs/1912.07242>.

- Preetum Nakkiran. More data can hurt for linear regression: Sample-wise double descent. arXiv:1912.07242, 2019b. URL <https://arxiv.org/abs/1912.07242>. arXiv preprint.
- Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. Deep Double Descent: Where Bigger Models and More Data Hurt, December 2019. URL <http://arxiv.org/abs/1912.02292>. arXiv:1912.02292 [cs, stat].
- Preetum Nakkiran, Prayaag Venkat, Sham Kakade, and Tengyu Ma. Optimal regularization can mitigate double descent (openreview entry). OpenReview forum submission, 2020. URL <https://openreview.net/forum?id=7R7fAoUygoa>.
- Preetum Nakkiran, Prayaag Venkat, Sham Kakade, and Tengyu Ma. Optimal regularization can mitigate double descent, 2021. URL <https://arxiv.org/abs/2003.01897>.
- Brady Neal. On the bias-variance tradeoff: Textbooks need an update, 2019. URL <https://arxiv.org/abs/1912.08286>.
- Jiawang Nie, Zhi Yang, and Guoyin Zhou. The saddle point problem of polynomials. *Foundations of Computational Mathematics*, 22(5):1133–1169, 2021. doi: 10.1007/s10208-021-09526-8. URL <https://link.springer.com/article/10.1007/s10208-021-09526-8>.
- Amanda Olmin and Fredrik Lindsten. Towards understanding epoch-wise double descent in two-layer linear neural networks, 2024. URL <https://arxiv.org/abs/2407.09845>.
- Liam Paninski. Statistics 4107: Intro to Math Stat (fall 2005), 2005. URL <https://sites.stat.columbia.edu/liam/teaching/4107-fall05/>.
- Mohammad Pezeshki, Amartya Mitra, Yoshua Bengio, and Guillaume Lajoie. Multi-scale feature learning dynamics: Insights for double descent, 2021. URL <https://arxiv.org/abs/2112.03215>.
- David Pfau. A generalized bias-variance decomposition for bregman divergences. Technical report, 2013.
- Victor Quétu and Enzo Tartaglione. Can we avoid Double Descent in Deep Neural Networks? In *2023 IEEE International Conference on Image Processing (ICIP)*, pp. 1625–1629, October 2023a. doi: 10.1109/ICIP49359.2023.10222624. URL <http://arxiv.org/abs/2302.13259>. arXiv:2302.13259 [cs].
- Victor Quétu and Enzo Tartaglione. Can we avoid Double Descent in Deep Neural Networks? In *2023 IEEE International Conference on Image Processing (ICIP)*, pp. 1625–1629, October 2023b. doi: 10.1109/ICIP49359.2023.10222624. URL <http://arxiv.org/abs/2302.13259>. arXiv:2302.13259 [cs].
- Sebastian Ruder. An overview of gradient descent optimization algorithms, June 2017. URL <http://arxiv.org/abs/1609.04747>. arXiv:1609.04747 [cs].
- Carl Runge. Über empirische funktionen und die interpolation zwischen äquidistanten ordinaten. *Zeitschrift für Mathematik und Physik*, 46:224–243, 1901. The original Runge example showing divergence with equispaced interpolation.
- Rylan Schaeffer, Mikail Khona, Zachary Robertson, Akhilan Boopathy, Kateryna Pistunova, Jason W. Rocks, Ila Rani Fiete, and Oluwasanmi Koyejo. Double Descent Demystified: Identifying, Interpreting & Ablating the Sources of a Deep Learning Puzzle, March 2023. URL <http://arxiv.org/abs/2303.14151>. arXiv:2303.14151 [cs, stat].
- Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, USA, 2014. ISBN 1107057132.
- Shai Shalev-Shwartz, Ohad Shamir, Nathan Srebro, and Karthik Sridharan. Learnability, stability and uniform convergence. *Journal of Machine Learning Research*, 11(90):2635–2670, 2010. URL <http://jmlr.org/papers/v11/shalev-shwartz10a.html>.
- Rahul Sharma and Alex Aiken. Bias-variance tradeoffs in program analysis. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '14*, pp. 127–137, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 978-1-4503-2544-8. doi: 10.1145/2535838.2535853. URL <https://doi.org/10.1145/2535838.2535853>.
- Zewen Shen and Kirill Serkh. On polynomial interpolation in the monomial basis. *SIAM Journal on Numerical Analysis*, 63(2):469–494, March 2025. ISSN 1095-7170. doi: 10.1137/23m1623215. URL <http://dx.doi.org/10.1137/23M1623215>.
- Cheng Shi, Liming Pan, Hong Hu, and Ivan Dokmanić. Homophily modulates double descent generalization in graph convolution networks, 2024. URL <https://arxiv.org/abs/2212.13069>.

- Jann Spiess, Guido Imbens, and Amar Venugopal. Double and single descent in causal inference with an application to high-dimensional synthetic control, 2023. URL <https://arxiv.org/abs/2305.00700>.
- Tom F. Sterkenburg. Statistical learning theory and occam’s razor: The core argument. *Minds and Machines*, 35(1), November 2024. ISSN 1572-8641. doi: 10.1007/s11023-024-09703-y. URL <http://dx.doi.org/10.1007/s11023-024-09703-y>.
- Masashi Sugiyama. *Introduction to Statistical Machine Learning*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2015. ISBN 9780128023501.
- Mark K. Transtrum, Gus L. W. Hart, Tyler J. Jarvis, and Jared P. Whitehead. egad! double descent is explained by generalized aliasing decomposition, 2025. URL <https://arxiv.org/abs/2408.08294>.
- L. G. Valiant. A theory of the learnable. *Commun. ACM*, 27(11):1134–1142, November 1984. ISSN 0001-0782. doi: 10.1145/1968.1972. URL <https://doi.org/10.1145/1968.1972>.
- Vladimir Vapnik. *The Nature of Statistical Learning Theory*. Springer: New York, 1999.
- Huiping Wang, Jinbo Xiong, Zhiqiang Yao, Mingwei Lin, and Jun Ren. Research survey on support vector machine. In *Proceedings of the 10th EAI International Conference on Mobile Multimedia Communications (MOBIMEDIA)*. EAI, December 2017. doi: 10.4108/eai.13-7-2017.2270596.
- Jingyuan Wang, Kai Feng, and Junjie Wu. Svm-based deep stacking networks. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI-19)*, pp. 5262–5268, 2019. doi: 10.24963/ijcai.2019/731. URL <https://www.ijcai.org/proceedings/2019/731>.
- Tian-Le Yang and Joe Suzuki. Dropout drops double descent. *Japanese Journal of Statistics and Data Science*, 7(2): 615–632, March 2024. ISSN 2520-8764. doi: 10.1007/s42081-024-00242-5. URL <http://dx.doi.org/10.1007/s42081-024-00242-5>.
- Zitong Yang, Yaodong Yu, Chong You, Jacob Steinhardt, and Yi Ma. Rethinking Bias-Variance Trade-off for Generalization of Neural Networks, December 2020. URL <http://arxiv.org/abs/2002.11328>. arXiv:2002.11328 [cs, stat].
- Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola. Dive into deep learning, 2023. URL <https://arxiv.org/abs/2106.11342>.
- Jiawei Zhang. Gradient Descent based Optimization Algorithms for Deep Learning Models Training, March 2019. URL <http://arxiv.org/abs/1903.03614>. arXiv:1903.03614 [cs].
- Rui Zhang and Hong Wang. The equivalence relationship between kernel functions based on svm and four-layer functional networks. In De-Shuang Huang, Vito Bevilacqua, Prashan Premaratne, and Praveen Gupta (eds.), *Intelligent Computing Methodologies*, volume 8589 of *Lecture Notes in Computer Science*, pp. 81–89. Springer, 2014. doi: 10.1007/978-3-319-09339-0_10. URL https://doi.org/10.1007/978-3-319-09339-0_10.
- Tong Zhang. *Mathematical Analysis of Machine Learning Algorithms*. Cambridge University Press, 2023. doi: 10.1017/9781009093057. URL <https://www.cambridge.org/core/books/mathematical-analysis-of-machine-learning-algorithms/EB9BABB05A5C312F19C38E5A01A5ECFC>.
- Ya Shi Zhang. Manipulating sparse double descent, 2024. URL <https://arxiv.org/abs/2401.10686>.

APPENDIX - GEMAN'S DERIVATION IN GEMAN ET AL. (1992)

Proof. We present the original derivation of bias-variance tradeoff and its analysis from Geman et al. (1992). Geman's paper is, by his own word, concerned of *parametric models*, i.e. hypothesis class without strong assumption of parameters defined, though the definition of bias-variance tradeoff afterward and its justification is made in the sense of parametric model. We partially clarify⁶ this with the following definition as customary.

Definition 7.3 (Parameterization). A **parametric model** is one that can be parameterized by a finite number of parameters. In general, for a hypothesis class $\mathcal{H}$ of all parametric hypothesis is then expressed as:

$$\mathcal{H} = \{f(x; \theta) : \theta \in \Theta \subset \mathbb{R}^d\} \quad (48)$$

where Θ is called the parameter space. A **nonparametric model** is one which cannot be parameterized by a fix number of parameters.

Suppose of a training dataset $\mathcal{D}$ of N 2-tuples $(\mathbf{x}_i, y_i)$, that is:

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\} \quad |\mathcal{D}| = N, \mathbf{x} \in \mathbb{R}^d, y \in \mathbb{R} \quad (49)$$

The regression problem is to construct a function $f(\mathbf{x}) \rightarrow y$ based on $\mathcal{D}$, which is denoted $f(\mathbf{x}; \mathcal{D})$ to show this dependency. Then,

$$\begin{aligned} \mathbb{E}[(y - f(\mathbf{x}; \mathcal{D}))^2 | \mathbf{x}, \mathcal{D}] &= \mathbb{E}[(y - \mathbb{E}[y | \mathbf{x}] + (\mathbb{E}[y | \mathbf{x}] - f(\mathbf{x}; \mathcal{D})))^2 | \mathbf{x}, \mathcal{D}] \\ &= \mathbb{E}[(y - \mathbb{E}[y | \mathbf{x}])^2 | \mathbf{x}, \mathcal{D}] + (\mathbb{E}[y | \mathbf{x}] - f(\mathbf{x}; \mathcal{D}))^2 \\ &\quad + 2\mathbb{E}[(y - \mathbb{E}[y | \mathbf{x}]) | \mathbf{x}, \mathcal{D}] \cdot (\mathbb{E}[y | \mathbf{x}] - f(\mathbf{x}; \mathcal{D}))^2 \\ &= \mathbb{E}[(y - \mathbb{E}[y | \mathbf{x}]) | \mathbf{x}, \mathcal{D}] + (\mathbb{E}[y | \mathbf{x}] - f(\mathbf{x}; \mathcal{D}))^2 \\ &\geq \mathbb{E}[(y - \mathbb{E}[y | \mathbf{x}])^2 | \mathbf{x}, \mathcal{D}] \end{aligned} \quad (50)$$

Hence, we decompose it to:

$$\mathbb{E}[(y - f(\mathbf{x}; \mathcal{D}))^2 | \mathbf{x}, \mathcal{D}] = \mathbb{E}[(y - \mathbb{E}[y | \mathbf{x}])^2 | \mathbf{x}, \mathcal{D}] + (f(\mathbf{x}; \mathcal{D}) - \mathbb{E}[y | \mathbf{x}])^2 \quad (51)$$

Where $\mathbb{E}[(y - \mathbb{E}[y | \mathbf{x}])^2 | \mathbf{x}, \mathcal{D}]$ is regarded to be a relative constant of variance on y given $\mathbf{x}$. Taking the expectation on $\mathcal{D}$, we gain:

$$\begin{aligned} \mathbb{E}_{\mathcal{D}}\left\{\mathbb{E}[(y - f(\mathbf{x}; \mathcal{D}))^2 | \mathbf{x}, \mathcal{D}]\right\} &= \mathbb{E}_{\mathcal{D}}\left\{\mathbb{E}[(y - \mathbb{E}[y | \mathbf{x}])^2 | \mathbf{x}, \mathcal{D}] + (f(\mathbf{x}; \mathcal{D}) - \mathbb{E}[y | \mathbf{x}])^2\right\} \\ &= \mathbb{E}_{\mathcal{D}}\left\{\mathbb{E}[(y - \mathbb{E}[y | \mathbf{x}])^2 | \mathbf{x}, \mathcal{D}]\right\} + \\ &= \mathbb{E}_{\mathcal{D}}\left\{(f(\mathbf{x}; \mathcal{D}) - \mathbb{E}[y | \mathbf{x}])^2\right\} \end{aligned} \quad (52)$$

The second term is of importance, and is further decomposed to:

$$\begin{aligned} &\mathbb{E}_{\mathcal{D}}\left\{(f(\mathbf{x}; \mathcal{D}) - \mathbb{E}[y | \mathbf{x}])^2\right\} \\ &= \mathbb{E}_{\mathcal{D}}\left\{(f(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})] + \mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})] - \mathbb{E}[y | \mathbf{x}])^2\right\} \\ &= \mathbb{E}_{\mathcal{D}}\left[(f(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})])^2\right] + \mathbb{E}_{\mathcal{D}}\left[(\mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})] - \mathbb{E}[y | \mathbf{x}])^2\right] + \\ &\quad 2\mathbb{E}_{\mathcal{D}}\left[(f(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})]) \cdot (\mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})] - \mathbb{E}[y | \mathbf{x}])\right] \\ &= \underbrace{\mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})] - \mathbb{E}[y | \mathbf{x}]}_{\text{bias term}} + \underbrace{\mathbb{E}_{\mathcal{D}}\left\{(f(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})])^2\right\}}_{\text{variance term}} \end{aligned} \quad (53)$$

which yields the desired bias-variance tradeoff. $\square$

⁶Generally, there is no definite definition on what would be considered non-parametric. Though, an example might be drawn from the (vanilla) neural network and Gaussian Process Regression (GPR)